

FocusGuard: Adaptive Transitions for Distraction-Aware Knowledge Work Using MR

Teamprojekt Dokumentation

von

Martin Berninger, Lars Ebner, Maximilian
Holdefer, Moritz Klock, Samuel Reinecker

Institut für Wirtschaftsinformatik (WIN)

Betreuer: Herr Luca Bernnardo

Betreuerin: Frau Chiara Krisam

Eingereicht am: 30. September 2026

Inhaltsverzeichnis

1	Projektablauf	1
1.1	Human Centered Design Process	1
1.2	HCP. 3.1 Planen und Verwalten	1
1.3	HCP. 3.2 Nutzungskontext	1
1.4	HCP. 3.3 Anforderungen	3
1.5	HCP. 3.4 Gestaltung	5
1.5.1	XR Origin	6
1.5.2	Passthrough	7
1.5.3	Ablenkungserkennung	7
1.5.4	Fokuserkennung	7
1.5.5	Objekterkennung	8
1.5.6	Virtueller Bildschirm	9
1.5.7	Debug-Menü	10
1.6	HCP. 3.5 Bewertung	10
2	Installation	13
2.1	Vorbereitung	13
2.2	Starten des Programmes	14
2.3	Kalibrieren der Webcam	16
3	Verwendung	19
3.1	Verschieben des Bildschirms	19
3.2	Ablenkungserkennung und Fokuseffekt	20
3.3	Objekterkennung	21
3.4	SteamVR-Fehler	22
4	Debug-Optionen	24
4.1	Verwenden des Debug-Menüs	24
4.2	Vorhandene Debug-Optionen	24
4.2.1	Show Camera	24

4.2.2	Play Video	25
4.2.3	Hide Detection Passthroughs	25
4.2.4	Show Environment	25
4.2.5	Move Keyboard	26
4.2.6	Alternative Movement	26
4.2.7	Record Measurements	26
4.2.8	Show Eye Tracking	26
4.2.9	Show Effect Strength	27
4.2.10	Freeze Effect Strength	27
4.2.11	Control Effect Strength	27
4.2.12	Move Webcam, Move Calibration Objects	27
4.2.13	Move Towel Roll, Move Water Bottle, Move Disinfectant	28
4.2.14	Optimize Webcam Rotation	28
4.3	Hinzufügen neuer Debug-Optionen	28
4.4	Standardwerte für Debug-Optionen	29
5	Systemarchitektur und Implementierung	31
5.1	Modul „Measurements“	32
5.1.1	Uniformes Listensystem	32
5.1.2	Ablenkungserkennung	33
5.1.3	Mikrosaccadenerkennung	33
5.1.4	Pupillengröße	34
5.1.5	Debug Recorder	34
5.2	Modul „Effects“	35
5.3	Implementierung des virtuellen Bildschirms	38
5.3.1	Absichten	38
5.3.2	Anpassung des Virtual-Monitors-Projektes	38
5.3.3	Bildschirmsteuerung	40
5.4	Objekterkennung	41
5.4.1	Architektur der Objekterkennung	42

5.4.2	Bildaufbereitung und YOLO-Inferenz	43
5.4.3	Detektionsergebnis und Konfiguration	46
5.5	Modul „Webcam“	48
6	Ausblick	51

1 Projektablauf

1.1 Human Centered Design Process

Für den Ablauf des Projektes hat sich das Team am Human Centered Design Process, kurz HCD, orientiert. Um den HCD-Prozess korrekt durchlaufen zu können, wurden zunächst die dazugehörigen Normen herbeigezogen, um sich einen besseren Überblick zu verschaffen und den weiteren Vorgang des Projektablaufs planen zu können. Nach Einblick in die Normen, insb. in die ISO 9241-220, die Prozessbeschreibungen zur menschenzentrierten Gestaltung enthält [5], wurde aufgrund des Umfangs und des Zeitlimits des Projektes entschieden, einen verkürzten HCD-Prozess anzuwenden. Die Teile HCP. 1 Sicherstellen, HCP. 2 Ermöglichen und HCP. 4 Einführung und Betrieb wurden ausgelassen und es wurde sich auf den Hauptteil, der in der Norm als HCP. 3 Durchführung bezeichnet ist, fokussiert. Der Abschnitt HCP. 3 Durchführung des HCD-Prozesses ist ein Kreislauf, bei dem zunächst mit einer Planung begonnen wird und danach die vier Schritte Nutzungskontext, Anforderungen, Gestaltung und Bewertung durchlaufen werden, bis das Projekt einen zufriedenstellenden Zustand erreicht hat. Für dieses Projekt hat das Team diesen Zyklus des HCD-Prozesses einmal durchlaufen.

1.2 HCP. 3.1 Planen und Verwalten

Zu Beginn des HCD-Prozesses hat sich jedes Teammitglied mit Hilfe der Normen zunächst selbstständig mit dem Ablauf der kommenden Schritte vertraut gemacht und Stichpunkte für die Abschnitte 3.2 und 3.3 aufgeschrieben. Diese wurden in den nächsten zwei Schritten als Grundlage verwendet. Zusätzlich wurde ein grober Zeitplan erstellt, um ein besseres Gefühl dafür zu erhalten, wie viel Zeit für jeden der kommenden Abschnitte zur Verfügung steht.

1.3 HCP. 3.2 Nutzungskontext

In diesem Schritt wurde zunächst ein vorläufiger Nutzungskontext erstellt, in welchem die Benutzergruppen, Ziele und Benutzungsumgebung des Systems in kurzer Form beschrieben wurden.

Ergebnisse:

Als Gegenstand der Nutzungskontextbeschreibung wurde hier festgehalten, dass es sich bei dem System um eine App zur Ablenkungsreduktion für die Mixed-Reality-Brille HTC Vive Focus Vision handelt. Diese soll die Realität, bei erkannter Ablenkung des Benutzers, langsam ausblenden, während wichtige Aspekte der Umgebung weiterhin eingeblendet

bleiben sollen. Die Ablenkungserkennung soll mit Hilfe von Eye-Tracking erfolgen, über das die gewählte Brille verfügt. Zu den Objekten, die immer eingeblendet bleiben sollen, gehören im Voraus ausgewählte Critical Anchors sowie Personen, die den Raum betreten.

Als Benutzergruppen wurden in diesem Schritt fünf Gruppen unterschieden. Schüler und Studenten, die im Klassenzimmer und Hörsaal oder alleine zuhause lernen, sowie Büroarbeiter, die im Büro oder zuhause arbeiten, wurden als die primären Benutzer des Systems angesehen. Zusätzlich wurde noch die Gruppe der Beeinträchtigten gelistet. Neben den Benutzern wurden auch noch zwei Gruppen von anderen Stakeholdern erfasst. Diese waren die Vorgesetzten, Dozenten, Lehrer und Eltern, die sich eine Steigerung der Produktivität der Benutzer durch Verwendung des Systems erhoffen, sowie die Mitarbeiter, Kinder, Haustiere und andere Personen in der Umgebung, die während der Benutzung weiterhin Aufmerksamkeit des Benutzers benötigen.

Als Ziele des Systems wurden die Verringerung von Produktivitätsverlusten, das Fördern der konzentrierten Arbeit sowie das Erkennen und Reduzieren von Ablenkungen ohne Verlust von wichtigen Umgebungsreizen vermerkt.

Als letzter Teil des vorläufigen Nutzungskontexts wurden sich Gedanken über die unterschiedlichen Umgebungen gemacht, in denen das System Verwendung finden könnte. Hierbei wurden Büro, Schule, Hörsaal und Zuhause im Homeoffice als die wohl häufigsten Umgebungen vermutet. Seltener könnte das System auch unterwegs in Cafés, der Bahn, dem Flugzeug oder anderen öffentlichen Räumen verwendet werden.

In der darauffolgenden Ausarbeitung des Nutzungskontexts wurden der Beschreibung des Systems die Vorbedingungen und Einschränkungen der Entwicklung hinzugefügt. Neben der limitierten Zeit des Projektes, welche bei der Entwicklung zu beachten war, wurde zusätzlich die Frage der Tester und deren Datenschutz aufgeworfen. Ein weiterer Punkt, der ein potenzielles Problem bei der Entwicklung werden könnte, war die Auflösung der Brille sowie die Genauigkeit ihrer Eye-Tracking Funktionen. Als Einschränkung für die spätere Nutzung des Systems wurden zusätzlich der Preis und die Akkulaufzeit der Brille und des Controllers vermerkt, und es entstanden Sorgen um die Sicherheitsprobleme, die durch die Ausblendung der Umgebung für Benutzer und Personen in deren Umgebung entstehen könnten.

Für die Benutzergruppen wurden zusätzliche Gruppen hinzugefügt, welche bei der Benutzung des Systems auf Probleme stoßen könnten. So haben viele Leute, die zum ersten Mal mit VR-Brillen interagieren, Beschwerden über Übelkeit und Motion Sickness. Eine weitere Personengruppe, die bei der Nutzung des Systems Schwierigkeiten haben könnte, sind Brillenträger, da zwar VR-Brillen auf Brillenträger ausgelegt sind, aber das Eye-Tracking, welches für die Erkennung von Ablenkungen und Fokus verwendet werden soll, möglicherweise durch die Brillengläser gestört werden könnte.

Bei der Zielsetzung wurde konkretisiert, dass bei dem Projekt nur ein grundlegender Prototyp des Systems erzeugt werden soll, der zur Veranschaulichung des Konzeptes dient. Dieser soll von möglichst vielen Nutzergruppen verwendet werden können und alle Grundfunktionen des Systems beinhalten. Diese Grundfunktionen sind das Erkennen der Ablenkungen und des Fokus mit Hilfe von Eye Tracking, die Ausblendung der Umgebung sowie die durchgängige Sichtbarkeit von Personen und Critical Anchors. Des Weiteren sollen die Barrierefreiheit und die schnelle Reaktion auf Reize in Echtzeit gewährleistet sein. Außerdem soll während und nach der Entwicklung des Systems der Datenschutz und andere Gesetze beachtet werden.

Als Abschluss des detaillierten Nutzungskontexts wurde der Teil Arbeitsaufgaben hinzugefügt. Dieser beschreibt, welche Schritte ein Benutzer durchlaufen muss, um das System verwenden zu können. Sobald der zukünftige Benutzer über die richtige VR-Brille verfügt, soll er die App herunterladen und installieren können. Nach einem Anmeldeprozess sollte er den Umgang mit dem Programm in einer Anleitung nachlesen können und dann direkt mit dem Arbeiten beginnen.

Nachdem nun der Nutzungskontext aufgestellt und die möglichen Benutzergruppen bekannt waren, wurden aus ihnen zur Veranschaulichung einige Personas erstellt. Dabei beschrieb jede Persona einen möglichen Benutzer durch die Punkte Name, Alter, Job und eine Beschreibung ihrer alltäglichen Aufgaben und Probleme. Es wurden die fünf Personas von Lily, Ben, Alois, Kai-Uwe und Carl erstellt, die Personen von unterschiedlichem Alter, Berufsgruppe und Lebensabschnitt mit Leben füllten.

1.4 HCP. 3.3 Anforderungen

Der nächste Schritt des HCD Prozesses war HCP 3.3 Anforderungen. Während bei dem Nutzungskontext die Benutzer beschrieben und klassifiziert wurden, wurde bei diesem Schritt festgehalten, was das System können muss. Dazu wurde hauptsächlich auf das FocusGuard-Paper [6] zurückgegriffen.

Ergebnisse:

Beim Start des Systems soll der Nutzer einige Critical Anchors festlegen können. Dies sind Objekte, die immer zu sehen sind, auch wenn die restliche Umgebung aufgrund einer erkannten Ablenkung ausgeblendet wird. Damit die ausgewählten Objekte weiterhin sichtbar sind, wird Objekterkennung eingesetzt, welche mit einem YOLO-Modell von Unity Sentis implementiert wird. Sobald das System läuft, sollen über Eye-Tracking Ablenkungen erkannt werden, welche bei Überschreitung von vorher definierten Schranken eine langsame Ausblendung der Umgebung auslösen sollen. Es werden zunächst zwei Schranken verwendet, einmal in Bezug auf die Länge der Ablenkungen und eine andere auf die

Menge von Ablenkungen in einer gewissen Zeitspanne. Die vorgesehenen Werte für die Schranken sind zehn Sekunden oder länger für eine einzelne Ablenkung oder mehr als fünf Ablenkungen in unter drei Sekunden. Als Ablenkung soll hier ein Wegschauen von der Region of Interest gelten, welche den Bildschirm sowie alle Critical Anchors beinhaltet. Falls kurz nach einer Überschreitung einer Ablenkungsschwelle und der noch während der Ausblendung der Umgebung eine Refokussierung festgestellt wird, soll das System direkt zur sichtbaren Umgebung zurückkehren. Sollte das System komplett in den Modus übergegangen sein, in dem die Umgebung ausgeblendet ist, so muss zuerst ein fokussierter Zustand gemessen werden, bevor die Ausblendung aufgelöst wird. Um einen Fokus wahrzunehmen, werden sowohl Mikrosakkaden als auch die Pupillengröße überwacht. Zusätzlich darf es in den letzten 60 Sekunden keine Überschreitung einer Ablenkungsschwelle geben.

Nach dem Erstellen der Anforderungen an das System wurde eine Befragung von potenziellen Nutzern in Form von semi-structured Interviews durchgeführt. Es wurden einige Leute angefragt und in einem Online-Interview zu ihren Meinungen über die Grundideen des Systems befragt. Es haben sechs Leute zu einem Interview zugesagt und haben eine Teilnehmerinformation sowie eine Datenschutzinformation erhalten. Die Interviews wurden jeweils von zwei Teammitgliedern geführt. Einer hat am Anfang das Projekt und den Ablauf des Interviews erklärt und dann während der Befragung Rückfragen und Bemerkungen für das Ende des Interviews notiert. Der Andere hat in dem Hauptteil des Interviews die Befragung durchgeführt. Die gesamten Interviews wurden aufgezeichnet und es wurde ein Transkript angefertigt. Diese Transkripte wurden anonymisiert und zusammengefasst, später wurden die wichtigsten Erkenntnisse und Anmerkungen aus den verschiedenen Interviews herausgeschrieben.

Ergebnisse:

In den Interviews wurden die Teilnehmer gefragt, ob, wie stark und wodurch sie beim Arbeiten abgelenkt werden. Als Antwort wurde mit großer Mehrheit berichtet, dass die Teilnehmer während der Arbeit oft abgelenkt werden oder den Fokus verlieren. Als häufigster Grund dafür wurde das Handy genannt, sei es wegen eingehender Nachrichten oder nur aus Verlockung etwas anderes zu machen. Als zweitgrößter Ablenkungsgrund wurden andere Personen, vor allem Mitarbeiter, Freunde und Familie genannt. Oft würden diese während der Arbeit zu den Teilnehmern kommen, um sie etwas zu fragen oder sich einfach nur zu unterhalten. Einige Teilnehmer erzählten zusätzlich, dass sie sich ab und zu dabei erwischten, von der Arbeit abzuschweifen und einfach in den Raum oder aus dem Fenster zu schauen, während andere bereits versuchten, Ablenkungen durch Weglegen des Handys oder der Nutzung von Einzelarbeitsräumen entgegenzuwirken. Somit bestätigen die Antworten der Teilnehmer die Vermutung, dass Ablenkungen bei der Arbeit für viele ein ständiger Begleiter sind. Während die meisten der Teilnehmer bisher keine oder

nur sehr beschränkte Erfahrungen mit VR-Brillen gemacht haben, stehen sie der Idee offen, eine Mixed-Reality-Brille während des Arbeitens zu verwenden, um die Menge der Ablenkungen zu reduzieren. Jedoch wurden dazu oft Sorgen über die Benutzungsdauer und das Schlechtwerden bei langer Benutzung einer VR-Brille geäußert. Zusätzlich gab es Bedenken, ob die Brille zu groß oder schwer wäre, um sie für mehrere Stunden am Tag zu tragen. Für die Benutzung des Systems kam während der Interviews von verschiedenen Teilnehmern der Wunsch auf, bei dem System selber über den Ausblendungszustand entscheiden zu können. So wollten einige selbst entscheiden können, ob eine gesamte Ausblendung der Umgebung geschehen soll oder wie viel der Umgebung noch sichtbar sein soll. Während dies verständliche Wünsche sind, gehen sie gegen die Grundidee des beabsichtigten Systems und wurden nur für zukünftige Weiterentwicklungen vermerkt. Zu dem tatsächlichen Ablauf einer Ausblendung gab es unter den Teilnehmern geteilte Meinungen. Manche bevorzugten eine schnelle Ausblendung, während andere dies als eigene Ablenkung ansahen. Während der eine Teilnehmer eine dauerhafte VR-Umgebung bevorzugte, nahm ein anderer die Vorstellung, in der echten Umwelt gar nichts mehr zu sehen, als bedrückend wahr. Einig waren sich die Teilnehmer jedoch, dass die Ausblendung möglichst fließend und natürlich ablaufen sollte, ohne zu viele Effekte, da sonst der Benutzer von der Ausblendung, die Ablenkungen verhindern und dem Benutzer beim Fokussieren helfen soll, selbst abgelenkt werden würde. Was die Teilnehmer bei der Ausblendung als zu ablenkend ansahen, unterschied sich. Somit wären dabei für die spätere Erweiterung des Systems einige Optionen sinnvoll.

1.5 HCP. 3.4 Gestaltung

Zusätzlich zu den Interviews wurde parallel mit Schritt 3.4 Gestaltung begonnen und ein einfacher Prototyp erstellt, welcher in der Zwischenpräsentation als Visualisierung der beabsichtigten Funktionen des endgültigen Systems dienen sollte. Dieser erste Prototyp enthielt eine Kamera und einen Controller, die sowohl mit einer VR-Brille als auch mit Maus und Tastatur bewegbar waren. Dazu wurde in Unity ein VR-Simulator verwendet, der Mausbewegungen in Kopfbewegungen umwandelte und Tastendrucke als Bewegungen des Nutzers interpretierte. Somit war es auch ohne Zugriff auf eine VR-Brille möglich, einfache Tests des Systems am PC durchzuführen. An der Kamera wurde ein Gaze Interactor befestigt, mit dem gemessen werden konnte, in welche Richtung der Benutzer gerade schaute. Anders als bei Eye-Tracking konnte hiermit jedoch nur die Richtung des Kopfes gemessen werden. Der Prototyp enthielt zusätzlich ein UI-Fenster, welches zwischen zwei Darstellungen wechseln konnte. Es wurde ein Unity-Skript erstellt, welches steuerte, wann das UI-Fenster die Darstellung änderte und gleichzeitig die Umgebungsfarbe langsam veränderte. Dadurch sollte die Ausblendung der Umgebung simuliert werden, ohne dass eine Umgebung vorhanden war, die ausgeblendet werden könnte. Um die verschie-

denen Methoden in dem Unity-Skript auszulösen, wurde ein Trigger an dem UI-Fenster befestigt, der auslöste, sobald der Gaze Interactor der Kamera mit ihm in Kontakt kam. In dem Skript wurde ein Wechsel des UI gestartet, sobald die Kamera für längere Zeit oder häufig in einem kurzen Zeitfenster von dem UI-Fenster wegschaute. Sobald wieder auf das UI-Fenster geschaut wurde, ging das UI in seinen Ausgangszustand zurück.

Nach der Zwischenpräsentation wurde mit der richtigen Entwicklung des FocusGuard-Systems begonnen. Es wurde entschieden, den vorläufigen Prototypen auszubauen und zu verbessern. Es wurde das OpenXR-Plugin, welches von den Herstellern der verwendeten Mixed-Reality-Brille bereitgestellt wird, in das Unity Projekt integriert und ein Repository auf GitHub erstellt, damit alle Teammitglieder gleichzeitig an dem System arbeiten konnten. Nach ein paar der ersten Commits, bei denen aufgrund von unterschiedlichen Unity-Versionen Errors und Probleme mit Paketen entstanden, wurde sich auf eine Unity Version geeinigt, mit der von diesem Punkt weitergearbeitet wurde. Dazu wurde die Version 6000.3.14f1 verwendet, da dies die vorliegende Version auf dem Institutsrechner war. Nachdem nun alle Teammitglieder in der Lage waren, zusammen an dem System zu arbeiten, wurden die verschiedenen Komponenten des Systems aufgeteilt und separat erstellt. Dabei hat sich Martin mit Unterstützung von Samuel um die Objekterkennung gekümmert und Lars hat zusammen mit Moritz den virtuellen Monitor implementiert. Samuel und Lars haben die Ablenkungs-, Mikrosakkaden- und Pupillenerkennungen gemacht, bei denen Moritz die Mikrosakkaden noch einmal verbessert und überarbeitet hat. Zusätzlich hat sich Moritz um die Zusammenführung und Architekturverbesserung der verschiedenen Abschnitte gekümmert, während Maxi sich um das Führen und Auswerten der Interviews und Workshops kümmerte.

1.5.1 XR Origin

Die Kamera des Prototypen wurde durch ein XR Origin ersetzt, welcher mithilfe eines Locomotion-Elementes korrekt auf die Brille reagierte und sich auf der richtigen Höhe zum Boden befand. An den Controllern wurden die Knöpfe der virtuellen Abbildungen mit den echten Controllern verbunden und die Kamera wurde um zwei Augen-Objekte ergänzt. Der Gaze Interactor an der Brille wurde ausgeschaltet und jedes der beiden Augen Objekte erhielt einen eigenen Gaze Interactor. Zusätzlich wurde von der Website der Entwickler der Vive Focus Vision der nötige Code für Eye-Tracking genutzt, um jedem Auge ein Skript zu geben, welches die Eye-Tracking Daten der Brille auf die entsprechenden Augen Objekte in Unity anwandte. Somit war nun ein funktionierendes Eye-Tracking erzeugt und das System konnte erkennen, wo der Benutzer zu jedem Zeitpunkt hinschaute. Neben der Blickrichtung der Augen wurde ebenfalls die Pupillengröße gemessen, welche später, zusammen mit den Richtungsdaten der Augen, für die Fokuserkennung genutzt wurde.

1.5.2 Passthrough

Der Hintergrund, der in dem Prototyp nur eine gleichfarbige Fläche war, welche ihre Farbe bei Ablenkung veränderte, wurde in ein richtiges Passthrough umgewandelt. Somit war es nun möglich, die reale Umgebung zu sehen und auszublenden. Jedoch lief die Implementierung des Passthrough nicht ohne Probleme ab. Zu Beginn der Implementierung stellte sich heraus, dass die Dokumentation auf der Website des Herstellers, wie Passthrough auf der ausgewählten Brille funktioniert, veraltet war und der Ablauf der Passthrough-Erstellung sich verändert hatte. Nach Durchsuchung der Vive Passthrough Bibliotheken wurde jedoch der korrekte Ablauf herausgefunden und es konnte ein Passthrough erzeugt werden. Nachdem der Passthrough funktionierte, wurde die Ausblendung durch Änderung der Intensität ermöglicht. Dabei wurden eine Vignette und eine Blur-Funktion hinzugefügt, die separat oder gemeinsam ablaufen konnten, um die Aufmerksamkeit des Benutzers wieder auf die Region of Interest zu fokussieren. Bei dem Versuch, den Ablauf der Ausblendung und deren aktuellen Stand anderen zu zeigen, stellte sich heraus, dass die VR-Brille aufgrund von Daten- und Privatsphärenschutz das Streaming verhindert, sobald ein Passthrough aktiv ist.

1.5.3 Ablenkungserkennung

Um das Passthrough zu aktivieren und eine Ausblendung der Umgebung zu starten, muss das System Ablenkungen erkennen. Hierzu wurde die Methode der Ablenkungszählung des Prototypen in ein komplexeres Listensystem umgewandelt, welches sich eine Architektur mit der Fokuserkennung teilte. Es wurde weiterhin dafür gesorgt, dass eine lange oder viele schnell aufeinanderfolgende kurze Ablenkungen zu einer Ausblendung führen.

1.5.4 Fokuserkennung

Die Fokuserkennung war in dem Prototypen nicht vorhanden und bestand nur daraus, dass der Benutzer erneut auf das UI-Fenster schaute. Für die neue Version war eine Fokuserkennung aus zwei Teilen geplant.

Der erste Teil war ein Algorithmus, der Mikrosakkaden erkennen sollte, um damit zu messen, ob der Benutzer sich tatsächlich auf etwas fokussiert. Hierzu wurden die Richtungsdaten des Eye-Trackings verwendet, um die Blickrichtung über einen Zeitraum zu speichern und später die gesammelten Daten auszuwerten. Wenn in einem gewissen Zeitfenster Mikrosakkaden erkannt wurden, so würde der Algorithmus dem System ein ok für die Reduktion der Ausblendung geben. Bei der Implementierung stellten sich allerdings einige Probleme dar. Aufgrund der Art, wie die VR-Brille die Eye-Tracking Daten an das System weitergab, wurde die Augenposition nur basierend auf der Framerate des Systems

aktualisiert. Da die Mikrosakkaden jedoch sehr schnell ablaufen und nur kurz anhalten, ist es nicht unwahrscheinlich, dass eine Mikrosakkade zwischen zwei Messungen stattfindet und somit von dem System übersehen wird. Zusätzlich war fragwürdig, ob die Präzision des Eye-Trackings überhaupt hoch genug war, um Mikrosakkaden wahrzunehmen, selbst wenn das System eine ausreichende Framerate hätte. So war das natürliche Rauschen, welches zu kleinen Änderungen in der gemessenen Augenrichtung führte, zwar so gering, dass es für normale Verwendung des Eye-Trackings keine Probleme erzeugen würde, jedoch von den geringen Zuckungen der Mikrosakkaden kaum zu unterscheiden war. Selbst nach einer Überarbeitung des Mikrosakkadenalgorithmus, die das Rauschen reduzieren sollte, erreichte das System aufgrund der Hardwarelimitierung keine verlässliche Möglichkeit, einen Fokus im Benutzer zu erkennen.

Der zweite Teil der Fokuserkennung war eine Überwachung der Pupillengröße des Benutzers. Da sich die Pupillen einer Person leicht weiten, wenn sich diese auf etwas fokussiert, war der Plan, mit Hilfe der Eye-Tracking-Daten diese Weitung zu erkennen und als Indiz eines Fokus zu bewerten. Anders als bei den Mikrosakkaden gab es bei den Pupillengrößen nicht das Problem, dass eine erfolgreiche Messung von der Framerate des Systems abhängt. Jedoch entstand bei diesem Teil ein ähnliches Problem wie zuvor, da auch hier eine geringe Änderung in den Augenwerten gemessen werden sollte, während diese Werte dauerhaft große Fluktuationen aufweisen. Da die Pupillengröße hauptsächlich von der Helligkeit des betrachteten Objekts und der Umgebung abhängt, gingen jegliche Größenänderungen, die von einem Fokus ausgehen könnten, in den Veränderungen aufgrund der Helligkeit unter. So war der Unterschied der Pupillengrößen bei Betrachtung eines hellen Fensters im Vergleich zu einem dunklen Fenster auf dem PC fast zwanzigmal so groß wie die erwartete Veränderung bei einem Fokus.

Schlussendlich wurden beide Teile der Fokuserkennung als unzuverlässig abgeschrieben und die Deaktivierung der Ausblendung wurde nur basierend auf der Blickrichtung ausgelöst.

1.5.5 Objekterkennung

Um Personen, die den Raum betreten, sichtbar zu machen, wurde Objekterkennung verwendet. Diese sollte die Videoaufnahmen der Brille verwenden, um darin die Personen im Blickfeld zu erkennen und ein partielles Passthrough über sie zu legen, damit selbst bei einer Ausblendung der Umgebung Personen für den Benutzer sichtbar sind. Ursprünglich sollte der Benutzer in der Lage sein, einige Personen als wichtig einzustufen, damit diese immer sichtbar sind, während alle anderen mit der Umgebung verschwinden. Im Umfang dieses Projektes wurde jedoch nur eine allgemeine Personenerkennung implementiert.

Zunächst wurde die Objekterkennung mittels eines YOLO-Modells von Unity Sentis auf

den Videoaufnahmen einer Laptopkamera in Unity zum Laufen gebracht und getestet. Als diese nun in die Brille integriert werden sollte, stellte sich heraus, dass die Brille aufgrund von Daten- und Privatsphärenschutz keinen Zugang zu ihrer Kamera zuließ. Dieses Problem wurde bereits bei dem Passthrough bemerkt, wo es ein Streamen der Benutzeransicht verhinderte, sobald ein Passthrough aktiv war. Für die Objekterkennung bedeutete dies, dass es nicht möglich war, die Personen in dem Blickfeld des Benutzers zu erkennen und somit einzublenden. Nach Absprache mit den Betreuern erhielt das Team eine separate Kamera, welche in den Raum und auf die Tür filmte und deren Aufnahme für die Objekterkennung verwendbar war. Kurzzeitig war der Plan, eine Benachrichtigung für den Benutzer einzublenden, wenn eine Person den Raum betrat und die Umgebung ausgeblendet war. Jedoch wurde durch ein virtuelles Raumverständnis ein Erlebnis ermöglicht, welches sehr nahe an die Ursprüngliche Idee heranreichte.

1.5.6 Virtueller Bildschirm

Damit der Benutzer nicht nur auf ein einfarbiges UI-Fenster schauen muss, wurde das Placeholder-UI vom Prototypen mit einem virtuellen Monitor ersetzt. Dazu wurde ein bestehendes Unity Projekt angepasst und erweitert, damit der Desktop des mit der Brille verbundenen PCs vor dem Benutzer im Raum als virtueller Bildschirm erscheint und in Position und Rotation angepasst werden kann. Es wurde ermöglicht, den Bildschirm nach oben und unten sowie zum Benutzer hin und weg zu verschieben. Zusätzlich konnte der Bildschirm um seine Längsachse rotiert und um den Benutzer rotiert werden. Bei der Anpassung des Unity Projektes an die VR-Brille ergab sich das Problem der Kompatibilität zwischen der Brille und dem vorliegenden Unity Projekt. Da das Ausgangsprojekt nur für PC-Anwendungen vorgesehen war, funktionierte es anfangs nicht, die Desktopaufnahme in der Brille abzubilden. Nach verschiedenen Versuchen wurde es geschafft, mit Hilfe von SteamVR eine funktionierende Verbindung zwischen der Brille und dem PC zu erzeugen, über die die Bildschirmaufnahme mit der Brille geteilt werden konnte. Allerdings benötigte die Aufnahme des Desktops, dass das Programm auf dem PC lief, und es kam zu dem Problem, dass die SteamVR Anwendung nach unterschiedlichen Zeiten zufällig abstürzte. Somit wurde, solange die Verbindung stabil blieb, eine Arbeitsumgebung geschaffen, auf die sich der Benutzer fokussieren konnte. Jedoch wurde schon nach kurzer Benutzung klar, dass mit dem System ein langanhaltendes Arbeiten wohl kaum möglich war, da die Auflösung der VR-Brille zu unscharf war, um Texte auf dem Bildschirm bei normalem Abstand zu lesen. Dies war auch der Fall bei dem Versuch, den echten Bildschirm zu verwenden.

1.5.7 Debug-Menü

Während der anderen Schritte wurden sowohl ein Script-Manager als auch ein Debug-Menü erstellt. Der Script-Manager ermöglichte eine einfache Auswahl, ob man auf der Brille oder auf dem PC das System testen wollte. Das Debug-Menü wurde für die Aktivierung und Deaktivierung von verschiedenen Funktionen der Brille erschaffen. Mit ihm wurden sowohl das Testen als auch Benutzerfunktionen möglich, die über die normale Verwendung hinausgingen. Es ermöglicht eine manuelle Kontrolle über die Ausblendung, das Anzeigen der Kameraaufnahmen, das Wechseln zu einer alternativen Steuerung sowie verschiedene andere Funktionen, die für Testzwecke nützlich waren. Die alternative Steuerung ändert, wie der Benutzer den virtuellen Bildschirm bewegen kann, da bei der ursprünglichen Steuerung Momente auftraten, bei denen die vorhandene Steuerung nicht ausreichend war. Da das System an einem PC mit einem sehr breiten Bildschirm getestet wurde, kam es zu dem Problem, dass der Benutzer den seitlichen Rand des Bildschirms nicht lesen konnte. Da die Auflösung der Brille zu schlecht war, um den Text auf dem Bildschirm bei normalem Abstand zu lesen, konnte man den Bildschirm an den Benutzer ziehen, jedoch blieb der Rand zu weit entfernt. Um dieses Problem zu lösen, wurde in der alternativen Steuerung ermöglicht, den Bildschirm vor dem Benutzer seitwärts zu bewegen, statt ihn um den Nutzer zu drehen. Somit war es möglich, den Rand des Bildschirms zentral vor sich zu haben, um den Text zu lesen.

1.6 HCP. 3.5 Bewertung

Schlussendlich kam das Projekt im letzten Abschnitt des HCD Zyklus an, dem HCP. 3.5 Bewerten. In diesem Schritt wurde auf den durchlaufenen Zyklus zurückgeblickt und ausgewertet, wie der Stand des Projekts ist.

In diesem Abschnitt hat das Team in einem Workshop, ähnlich wie in den Interviews, das System potenziellen Benutzern vorgestellt und ihr Feedback zu den Vor- und Nachteilen des Systems eingeholt. Anders als bei den Interviews konnten die Teilnehmer in den Workshops das System auf der VR-Brille selbst testen und sich einen eigenen Eindruck davon machen, wie es ist, damit zu arbeiten. Um sicherzugehen, dass die Teilnehmer die Funktionen des Systems vollständig erleben konnten, hat das Team einen Ablaufplan für den Workshop erstellt.

Der Workshop begann mit einer Einführung in das Projekt. Dabei wurde den Teilnehmern zunächst eine Erklärung des Problems sowie eine knappe Vorstellung des Konzeptes des Systems gegeben. Ihnen wurde erklärt, wie die VR-Brille mittels Eye-Tracking und Ausblendung bei dem Problem der Ablenkungen helfen soll und wie das System grob funktioniert. Hier sollten die Teilnehmer kurz sagen, was sie von so einem System erwar-

ten und wie ihre Vorstellungen aussehen, damit man ihre Meinung vor und nach dem Test vergleichen kann. Anschließend wurde ihnen erklärt, wie man mit dem System arbeitet und wie man es bedient. Es wurde das Eye-Tracking der Brille an den Teilnehmer angepasst und dafür gesorgt, dass das System richtig läuft. Daran anschließend sollten die Teilnehmer einige Minuten Tetris spielen und danach eine Weile einen Tipptest machen und als Letztes einen Text lesen. Derweil wurden Ablenkungen durch das Smartphone, das Betreten des Raumes, Gespräche und Bewegung simuliert, um die Features des Systems zu testen. Um beide Steuerungsvarianten des Systems zu berücksichtigen und zu testen, wurde bei manchen Teilnehmern mit der einen, bei anderen mit der anderen Variante gestartet. In der Mitte des Tests wurde die Steuerung umgeschaltet, damit jeder Benutzer beide Varianten ausprobieren und anschließend seine Präferenzen abgeben konnte. Nach dem Test wurden die Teilnehmer über die Funktionen des Prototypen befragt und als Abschluss des Workshops sollten sie noch einmal ihre Meinung zu dem gesamten System teilen. Es wurde gefragt, ob das System ihre Erwartungen erfüllt oder verändert hat, was gut funktionierte und an welchen Stellen Verbesserungen benötigt werden.

Ergebnisse:

An den Evaluationsworkshops nahmen Personen teil, welche vor dem Projekt keine nennenswerten Erfahrungen mit VR-Brillen gesammelt hatten. Vor der Nutzung des Prototyps äußerten sie die Erwartung, dass das System schnell und zuverlässig reagieren und für die Arbeit nicht benötigte Teile der Umgebung ausblenden sollte. Gleichzeitig sollten relevante Bereiche wie der Arbeitsplatz oder andere zum Arbeiten benötigte Bereiche weiterhin sichtbar bleiben. Die Veränderung der Umgebung sollte dabei möglichst unauffällig erfolgen, sodass die Ausblendung natürlich abläuft, sodass sie nicht selbst direkt eine Ablenkung darstellt.

Nach der Benutzung der Brille bewerteten die Teilnehmenden das System grundsätzlich positiv und schätzten es als hilfreich ein, Ablenkungen während der Arbeit zu reduzieren. Die Zeit, bis eine Ablenkung eine Ausblendung bewirkt, wurde als passend empfunden. Die erneute Einblendung der Umgebung bei einer Fokussierung hingegen wurde als langsam empfunden, wobei sie sich gerne eine schnellere Rückkehr in die Realität gewünscht hätten. Nach Anpassung der Zeit von 60 Sekunden auf 5 Sekunden wurde diese allerdings auch als passend bewertet. Bei der Ausblendung wurde der Übergang zu einer hellen, in diesem Fall weißen, Umgebung als angenehm bewertet. Eine Ausblendung durch einen Blur Effekt wurde als weniger passend bewertet, da die Teilnehmer es als ablenkend und irritierend einschätzten, weiterhin etwas Unscharfes wahrnehmen zu können, aber nicht zu sehen, was sich dort befindet. Ähnlich war die Meinung zu dem Vignetteneffekt, der durch seine schwarze Präsenz am Blickfeldrand teilweise als bedrückend empfunden wurde.

Die Steuerung des Bildschirms wurde von den Teilnehmern grundsätzlich als intuitiv bezeichnet, wobei die ursprüngliche Steuerung, bei welcher der virtuelle Monitor um den

Benutzer drehbar ist, bevorzugt wurde. Jedoch fanden sie auch die Funktion der Seitwärtsbewegung der alternativen Steuerung als hilfreich, wobei sie eine Steuerung bevorzugten, die über beide Funktionen verfügte. Ebenso wurde bei der alternativen Steuerung die horizontale und vertikale Bewegung des Bildschirms auf einem Joystick vereint gewünscht.

Als größte technische Einschränkung wurde die geringe Auflösung des Bildschirms genannt, sodass es den Teilnehmenden schwer fiel, Text und kleinteilige Elemente auf dem Bildschirm zuverlässig zu lesen beziehungsweise zu erkennen. Diese Unschärfe ist jedoch auf die Brille selbst und nicht auf den Prototypen zurückzuführen. Bei den beiden anderen Aufgaben, die einen Tipptest und Tetris umfassten, wurde die Unschärfe als weniger störend bewertet.

Ein zentrales gemeinsames Ergebnis war der Wunsch nach mehr Kontrolle und individuellen Anpassungsmöglichkeiten. Da bei dem Test, neben Bildschirm und Personen, nur die Tastatur und Maus sichtbar waren, entstand der Wunsch, den gesamten Schreibtisch eingeblendet zu lassen. Die Teilnehmenden wollten ebenso die Möglichkeit haben, den eingeblendeten Bereich und was dieser umfasst, selbst auswählen zu können. Weitere Wünsche umfassten die Fähigkeit, bereits ausgeblendete Teile der Umgebung separat wieder einzublenden, sowie die manuelle Möglichkeit, zwischen Ausblendung und Passthrough zu wechseln. Auch beim Aussehen der Ausblendung gab es individuelle Vorlieben und es wurde ein individualisierbarer Hintergrund gewünscht. Ein Teilnehmer schlug sogar vor, am Anfang der Benutzung eine Auswahl zu integrieren, bei der zwischen verschiedenen Modi der Ausblendung ausgewählt werden kann.

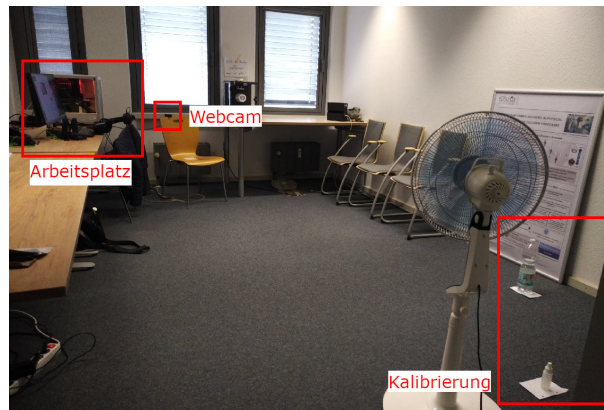


Abbildung 1: Aufbau im VR-Labor während des Projektes

2 Installation

Das Programm wird direkt aus Unity heraus ausgeführt. Dazu sind folgende Voraussetzungen notwendig:

- PC mit Windows 11
- Vive Focus Vision Pro
- Jabra PanaCast 20 bzw. Webcam mit 90° horizontalem Field of view (FOV)
- Unity (Version 6000.3.141f)
- SteamVR (Version 2.16.7)
- Vive Hub (Version 2.5.5)

2.1 Vorbereitung

Zunächst muss der Raum, in dem das Projekt in Betrieb genommen werden soll, vorbereitet werden. Die Benutzer muss dafür an einem Arbeitsplatz mit der Tastatur vor ihm sitzen. Die Tastatur sollte gut ausgeleuchtet sein.

Die Kamera muss so hinter dem Benutzer montiert werden, dass der Benutzer auf dem Bild nicht zu sehen ist. Um die spätere Kalibrierung zu erleichtern, sollte sie waagrecht montiert sein. Idealerweise sollte sie auf eine Ecke des Raumes ausgerichtet sein.

Der Aufbau, der während unseres Projektes verwendet wurde, ist in Abb. 1 dargestellt. Abb. 2 stellt die Positionierung der Kamera genauer dar.

Auto-Zoom muss bei der Kamera deaktiviert sein. Dies geht in den Windows-Einstellungen unter „Bluetooth und Geräte > Kameras > »Name der Kamera«, indem dort ein festes Zoom-Level eingestellt wird.

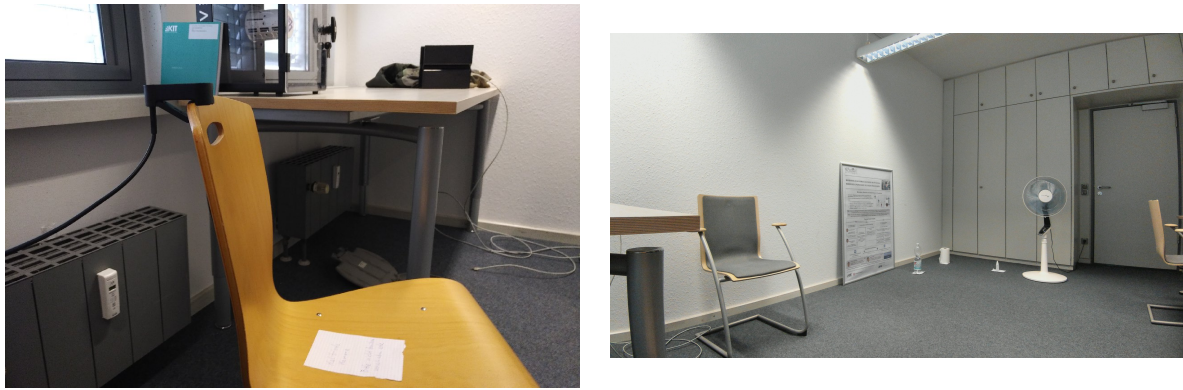


Abbildung 2: Links: Waagerechte Position der Webcam auf einem Stuhl. Rechts: Das VR-Labor aus Sicht der Webcam.

Auf der Brille muss die Begrenzung so gezogen werden, dass der Benutzer an seinem Arbeitsplatz genau in der Mitte der Begrenzung sitzt und dabei nach vorne schaut, wenn er auf den Bildschirm blickt. Außerdem sollte „Handverfolgung“ in den Einstellungen der Brille deaktiviert werden, da dies dazu führt, dass Handbewegungen unbeabsichtigt Controller-Tasten auslösen.

Damit die Webcam später kalibriert werden kann, müssen bestimmte Markierungen auf dem Fußboden platziert werden. Hierbei handelt es sich um drei Punkte, die ein rechtwinkliges Dreieck mit zwei Katheten der Länge 70 cm bilden, wobei die Hypothense der Kamera am nächsten gelegen ist (vgl. Abb. 3). Die Punkte sollten möglichst mittig im Bild der Kamera liegen, d.h. in der Ecke des Raumes, auf die die Kamera zeigt.

Diese Punkte werden zusätzlich mit zylinderförmigen Objekten markiert (vgl. Abb. 4). Dabei sollten folgende Dimensionen verwendet werden, da die entsprechenden Objekte in Unity diese Größe verwenden, um die Kalibrierungsobjekte zu modellieren:

- Der am weitesten entfernte Punkt (rot): 11,5 cm Durchmesser, 19,5 cm Höhe
- Linker Punkt (blau): 8,5 cm Durchmesser, 34 cm Höhe
- Rechter Punkt (grün): 4,5 cm Durchmesser, 12 cm Höhe

2.2 Starten des Programmes

Der Quellcode des Projektes befindet sich in einem öffentlichen Projekt auf GitHub¹ und muss von dort geklont werden. Anschließend kann das Projekt geöffnet und dort die Szene „Assets > Scenes > Focus“ in die Objekthierarchie gezogen, aber noch nicht gestartet werden. Die Standardszene kann gelöscht werden.

¹<https://github.com/LarsEbner/FocusGuard>

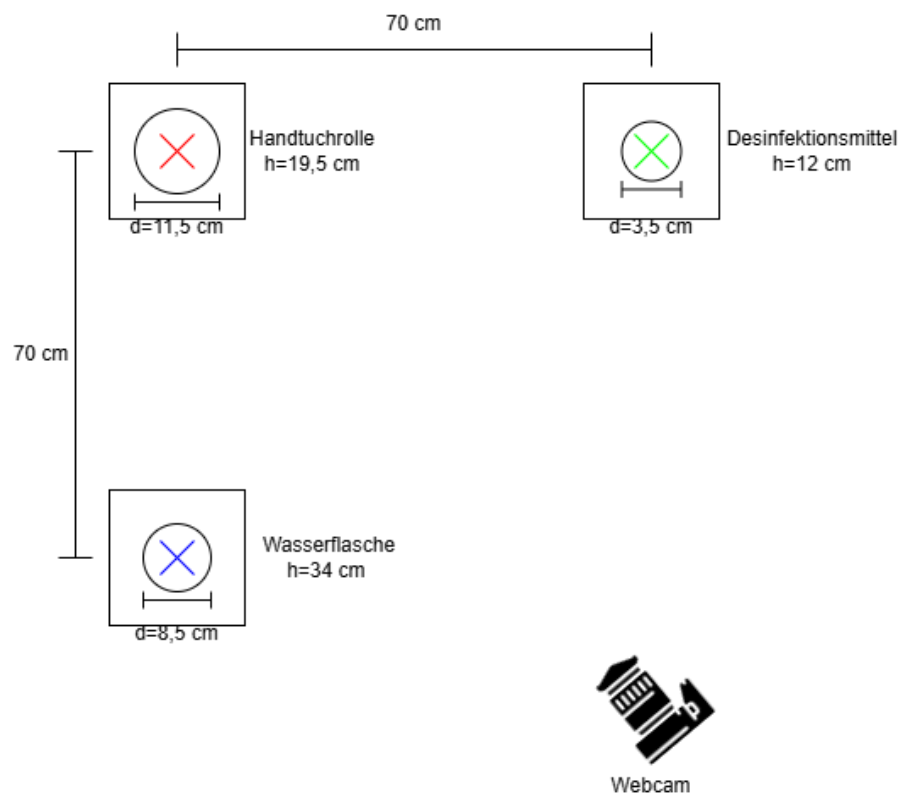


Abbildung 3: Schematische Darstellung der Kalibrierungspunkte



Abbildung 4: Kalibrierungspunkte aus Sicht der Kamera

Die Brille muss nun eingeschaltet werden. Anschließend kann auf dem PC das Programm „Vive Hub“ und anschließend „SteamVR“ gestartet werden (in dieser Reihenfolge). Daraufhin kann in den Einstellungen der Brille die PC-Verbindung aktiviert werden. In den Spielen auf der Brille sollte nun „SteamVR“ erscheinen (dafür die Spieleansicht oben rechts auf „PCVR-Apps“ umschalten). Sobald man auf der Brille SteamVR gestartet hat, sollte es möglich sein, im SteamVR-Fenster auf dem PC das gestreamte Bild von der Brille anzusehen.

Wichtig: „Vive Hub“ startet nicht, wenn auf dem PC schon „Vive Business Streaming“ läuft. Während der Entwicklung hatten wir das Problem, dass dieses Programm von einem anderen Benutzer auf dem Windows-PC automatisch gestartet wurde und wir ohne Adminrechte keine Möglichkeit hatten, dieses zu beenden, ohne den PC neu zu starten. Am PC im VR-Labor wurde der Autostart dieses Programmes mittlerweile deaktiviert.

Auf dem PC muss außerdem darauf geachtet werden, dass kein anderes Programm (z.B. die Kamera-App von Windows) aktuell die Webcam verwendet.

Nun muss die Brille aufgesetzt werden und darauf gewartet werden, dass die Kalibrierung des Augenabstandes durchläuft. Danach kann das Programm über den „Play“-Button in Unity gestartet werden².

Falls man die Brille während des Startens des Programmes noch nicht aufgesetzt hat, kann es gelegentlich vorkommen, dass weder das Passthrough sichtbar ist noch das Eye-Tracking Werte liefert. Sollte dieser Fehler auftreten, reicht es, das Programm in Unity neu zu starten (während man die Brille trägt).

2.3 Kalibrieren der Webcam

Damit die gefundenen Objekte aus der Objekterkennung zuverlässig in den Raum projiziert werden können, muss die Webcam kalibriert werden, d.h. es müssen Bodenhöhe sowie Position und Rotation der Webcam bestimmt werden. Dies ist vollständig aus dem Programm heraus möglich und kann auch nach dem ersten Start erfolgen.

Die kalibrierten Positionen werden beim Beenden des Programms automatisch im Projektordner gespeichert und beim Starten des Programms automatisch geladen. Auf dem PC im VR-Labor liegt sie in einer Datei unter folgendem Pfad:

`„C:\Users\Studis\AppData\LocalLow\DefaultCompany\FocusGuard\focusguard.json“`

²In der Release-Sektion auf GitHub existiert auch ein Build der Anwendung für Windows (Datei „bin.zip“). Dieser kann heruntergeladen und ausgeführt werden, anstatt das Repository zu klonen. Beim Starten öffnet sich Unity im Vollbildmodus und verdeckt den Desktop, daher muss „Windows+D“ gedrückt werden, um diesen wieder anzuzeigen. Da das Programm aber lediglich ein Prototyp ist, der nicht produktiv verwendet werden kann, sollte besser gleich der Quellcode heruntergeladen werden.

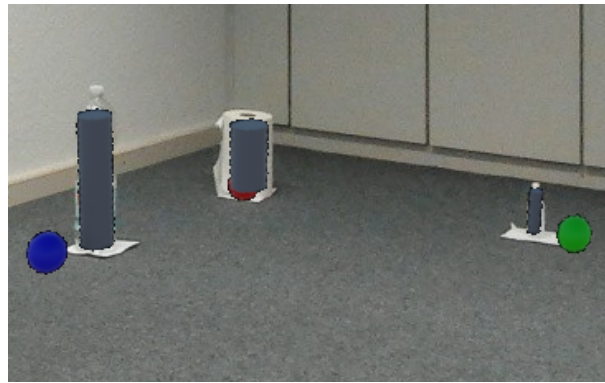


Abbildung 5: Kalibrierungsobjekte in der VR-Welt mit projizierten Kalibrierungspunkten. Das Passthrough wurde per Bildbearbeitung nachträglich hinzugefügt.

Auf anderen Rechnern müsste der Name des Benutzers ersetzt werden. Es können bestehende Konfigurationen für den Raum kopiert und gespeichert werden.

Das Git-Repository enthält eine Datei namens „`VR-Labor.json`“, die die Konfiguration des Raumes enthält, der während der Entwicklung der Anwendung verwendet wurde. Im einfachsten Fall reicht es also aus, das Programm wieder zu beenden, dann die „`focusguard.json`“ durch diese Datei vollständig zu ersetzen, und anschließend das Programm wieder zu starten.

Eine vollständige Neukonfiguration des Raumes ist deutlich aufwendiger. Sie erfolgt über Debug-Optionen (ein- und ausschaltbar über die Schultertasten, siehe 4.1).

Zunächst muss die Option „Show Camera“ angeschaltet werden, damit sowohl das Webcambild mit den dort eingezeichneten Kalibrierungspunkten (vgl. Abb. 4) als auch die Position der Webcam und der Kalibrierungsobjekte in Form eines grauen Quaders bzw. grauen Zylinders sichtbar sind (vgl. Abb. 5).

Die Aufgabe der Kalibrierung besteht darin, die Position der Kamera und der Objekte genau zu bestimmen und anschließend die Kalibrierungspunkte im Webcambild in Abb. 4 einzuzichnen, sodass eine Projektion der eingezeichneten Punkte in den Raum möglichst nahe an den tatsächlichen Positionen der Kalibrierungspunkte an der Unterseite der Objekte liegt. Kleine Abweichungen wie in Abb. 5 sind dabei unschädlich und insbesondere an den Rändern des Webcambildes zunehmend zu erwarten. Womöglich werden sie hier durch eine nicht ganz korrekte Modellierung des FOV der Webcam ausgelöst.

Die Webcam sowie die Kalibrierungsobjekte können verschoben werden, indem im Debug-Menü die Optionen „Move Webcam“ bzw. „Move Calibration Objects“ aktiviert werden. Alle drei Kalibrierungsobjekte werden dabei zusammen verschoben. Mit dem linken Thumbstick lassen sich die Objekte in horizontaler Richtung verschieben, mit dem rechten Thumbstick nach oben und unten bzw. um die vertikale Achse drehen. Die Optionen sollten nach dem Verschieben wieder deaktiviert werden, da sie das Verschieben des virtuellen

Monitors unterbinden.

Anschließend müssen über die Optionen „Move Towel Roll“, „Move Water Bottle“ und „Move Disinfectant“ die Kalibrierungspunkte wie in Abb. 4 im Webcambild eingezeichnet werden. Hierbei verschiebt man alle Punkte einzeln mit dem linken Thumbstick. Auch hier sollten die Optionen im Anschluss deaktiviert werden.

Erst wenn die Objekte korrekt platziert und alle Punkte eingezeichnet sind, kann die letzte Debug-Option, „Optimize Webcam Rotation“, eingeschaltet werden. Diese rotiert die Webcam automatisch so, dass der Abstand zwischen den Projektionen der eingezeichneten Punkte und den tatsächlichen Positionen an der Unterseite der Objekte minimiert wird, wodurch ein Ergebnis wie in Abb. 5 entstehen konnte. Damit ist die Kalibrierung abgeschlossen.

Während der Entwicklung hatten wir teilweise das Problem, dass sich die Kamera in einer Position befand, in der diese automatische Optimierung nicht mehr funktionierte. Dies war der Fall, nachdem wir die Auflösung des Kamerabildes verändert hatten und danach versehentlich die Optimierung gestartet hatten, obwohl wir die eingezeichneten Positionen der Kalibrierungspunkte im Webcambild noch nicht angepasst hatten. Dadurch geriet die Kamera in eine Position, in der die eingezeichneten Kalibrierungspunkte nicht mehr auf dem Fußboden liegen, weshalb es für die Optimierung nicht möglich war, sie zu projizieren, und damit die initiale Abweichung für die Optimierung zu bestimmen.

Gelöst werden musste das Problem, indem in der Datei „`focusguard.json`“ das JSON-Objekt mit der ID „`webcam-camera-object`“ gelöscht wurde. Damit wurde nach einem Neustart des Programmes die Rotation der Kamera auf den Standardwert in Unity zurückgesetzt, was ausreichend war, um die Optimierung erfolgreich zu starten.

Eine mögliche Verbesserung, um dieses Problem künftig zu umgehen, könnte darin bestehen, dass die Kamera zu Beginn der Optimierung so gedreht wird, dass sie genau auf das erste Kalibrierungsobjekt zeigt.

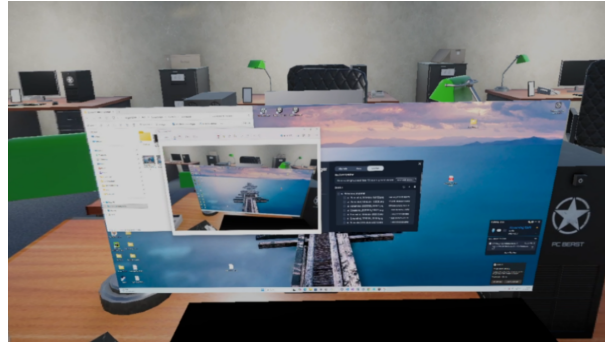


Abbildung 6: Der Bildschirm in der virtuellen Umgebung. Der schwarze Quader unten im Bild ist eigentlich ein Passthrough für die Tastatur.

3 Verwendung

Dieser Abschnitt beschreibt die gewöhnliche Verwendung des Programms.

Nach dem Starten des Programms sieht man seinen PC-Bildschirm als virtuellen Monitor gespiegelt vor sich. Zudem sieht man seine aktuelle Umgebung als Passthrough von der VR-Brille. Leider verhindert die VR-Brille von VIVE, dass man Aufnahmen oder Screenshots machen kann, falls das Passthrough aktiviert ist. Daher verwenden die meisten Screenshots in dieser Dokumentation stattdessen eine vollständige VR-Umgebung (z.B. in Abb. 6)³.

3.1 Verschieben des Bildschirms

Der Benutzer kann den virtuellen Bildschirm über die Thumbsticks der beiden Controller verschieben.

Wird der linke Thumbstick nach links oder rechts gedrückt, wird der Bildschirm entlang eines Kreises verschoben, dessen Mittelpunkt der Koordinatenursprung bildet, an dem sich der Benutzer befindet. Dies ist in Abb. 7 dargestellt. Dort sieht man ebenfalls, dass der Bildschirm nach vorne und nach hinten verschoben werden kann, wofür man den linken Thumbstick nach vorne oder nach hinten drücken muss. Der Bildschirm wird dabei automatisch so rotiert, dass er in Richtung des Benutzers zeigt.

Drückt man den rechten Thumbstick nach vorne oder nach hinten, wird der Bildschirm nach oben oder nach unten verschoben. Drückt man den rechten Thumbstick nach links oder nach rechts, wird der Bildschirm gekippt.

³Kurz vor der Abgabe des Projektes wurde festgestellt, dass es in den Einstellungen des Headsets eine Option gibt, um Aufnahmen bei eingeschaltetem Passthrough zu ermöglichen [1]

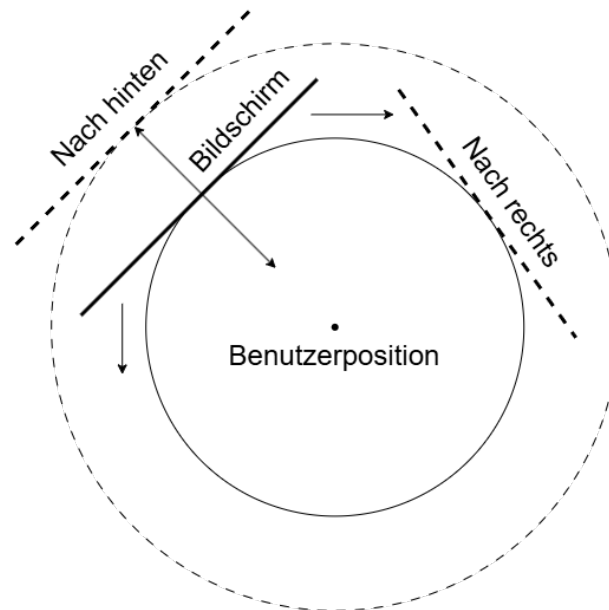


Abbildung 7: Schematische Darstellung der Bildschirmverschiebung

3.2 Ablenkungserkennung und Fokuseffekt

Die Anwendung versucht einzuschätzen, ob der Benutzer aktuell von seiner Arbeit abgelenkt ist. Dazu verwenden wir das Konzept einer Region of Interest (ROI). Wenn der Benutzer nicht auf ein Objekt der ROI schaut, wird dies als Indiz für Ablenkung gewertet.

Zur ROI zählen folgende Objekte:

- der virtuelle Bildschirm
- das Passthrough für die Tastatur
- die Passthrough-Zylinder der Objekterkennung (siehe 3.3)

Eine Ablenkung wird erkannt, falls der Benutzer entweder für mehr als zehn Sekunden aus der ROI wegschaut, oder der Benutzer innerhalb der letzten drei Sekunden mindestens fünfmal aus der ROI weggeschaut hat.

Es werden zusätzlich weitere Daten zu Augenbewegungen und Pupillengröße erfasst, allerdings werden diese gegenwärtig nicht für die Ablenkungserkennung einbezogen.

Wurde eine Ablenkung erkannt, setzt der Fokuseffekt ein, damit sich der Benutzer wieder auf seine Arbeit konzentriert. Die vollständige Transition dauert zehn Sekunden und ist in Abb. 8 dargestellt. Innerhalb der zehn Sekunden wird das Umgebungspassthrough langsam ausgeschaltet. Dies macht sich dadurch bemerkbar, dass die Umgebung immer dunkler wird. Nach fünf Sekunden wird zudem eine weiße Hintergrundfarbe über das Passthrough gelegt, sodass man am Ende nur noch eine weiße Umgebung sieht. Außerdem

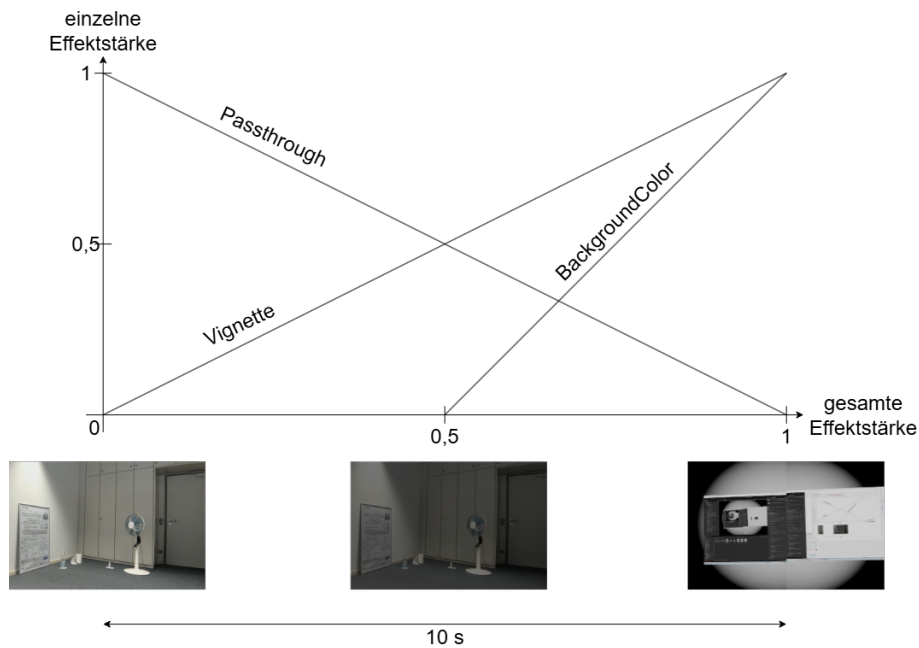


Abbildung 8: Schematische Darstellung des Fokuseffektes

erscheint während der gesamten Transition eine Vignette, die man jedoch erst zum Schluss wahrnimmt.

Da der weiße Hintergrundeffekt nicht sichtbar ist, wenn die vollständige VR-Umgebung eingeschaltet ist, wird auf den Bildern in dieser Dokumentation stattdessen ein Blur-Effekt eingesetzt. Dieser ist allerdings nicht sichtbar, wenn das normale Umgebungspassthrough verwendet wird.

Der Fokuseffekt wird wieder entfernt, nachdem für die Dauer von 60 Sekunden keine weitere Ablenkung nach den beiden Regeln festgestellt wurde (und der Benutzer aktuell auf die ROI schaut, andernfalls könnte es sein, dass kurz nach dem Umschalten erneut eine Ablenkung nach der 10-Sekunden-Regel festgestellt werden würde, falls der Benutzer schon für mehrere Sekunden aus der ROI wegschaut). Dabei läuft der oben beschriebene Fokuseffekt in umgekehrter Reihenfolge ab.

3.3 Objekterkennung

Für die Objekterkennung wird das Bild der Webcam ausgewertet, indem vorhandene Personen markiert werden. Dem YOLOv8-Modell, das für die Objekterkennung verwendet wird, gelingt dies zuverlässig. Es wäre noch in der Lage, weitere Objektklassen zu erkennen, allerdings haben wir bei den Klassen, die wir kurz eingeschaltet hatten, festgestellt, dass dies entweder gar nicht (bei Büchern) oder nur sehr unzuverlässig (bei Stühlen) funktioniert hat.

Wird eine Person erkannt, wird an dieser Stelle im Raum ein zylinderförmiges Passthrough



Abbildung 9: Passthrough mit erkannter Person bei vollem Fokuseffekt (Passthrough per Bildbearbeitung nachträglich hinzugefügt)

erstellt. Damit dieser auch tatsächlich die erkannte Person umschließt, muss die Webcam gut genug kalibriert sein. Das Passthrough bewegt sich mit der Person mit. Es ist natürlich nur vom Hintergrund-Passthrough unterscheidbar, falls der Fokuseffekt aktuell aktiviert ist (siehe Abb. 9). Das Passthrough ist selbst wieder eine ROI, sodass durch das Anschauen einer Person der Fokuseffekt nicht ausgelöst bzw. wieder zurückgesetzt wird.

3.4 SteamVR-Fehler

Bei der Nutzung des Programmes tritt unregelmäßig ein Fehler auf, der sich dadurch bemerkbar macht, dass sich der Benutzer nicht mehr im VR-Raum frei bewegen kann, sondern „durch den Raum schwebt und sich dabei dreht“, und zudem auf der VR-Brille hin und wieder ein „Flimmern“ (siehe Abb. 10) sowie einen Ladebildschirm mit VIVE-Logo sieht.

Das Problem hat wohl einen Zusammenhang mit SteamVR: Der Fehler trat nicht auf, als wir die Anwendung noch direkt auf der Brille ausgeführt hatten, und er tritt weiterhin im SteamVR-Menü auf der Brille auf, wenn man die Anwendung in Unity beendet.

Sollte das Problem auftreten, muss man auf der Brille SteamVR beenden und erneut starten. Die Anwendung kann dabei in Unity weiterlaufen.

Da der Fehler spontan bei der normalen Benutzung auftritt, konnten wir leider keine genaue Ursache herausfinden. Auch zur Häufigkeit können wir keine genauen Angaben machen: Zum Teil tritt der Fehler mehrfach innerhalb weniger Minuten auf, wir konnten die Anwendung aber auch schon über eine Dreiviertelstunde am Stück nutzen, ohne dieses Problem zu erhalten. Vielleicht ist der Fehler daher auf eine Überlastung der Brille durch unsere Anwendung zurückzuführen.

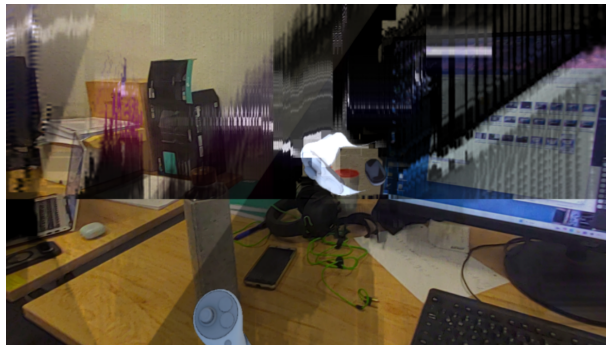


Abbildung 10: Typische Ansicht auf der Brille beim Auftreten des SteamVR-Fehlers

4 Debug-Optionen

Viele Funktionen des Programms lassen sich nur sinnvoll testen, indem man das Programm direkt ausführt. Dabei wurden während der Entwicklung immer wieder Hilfsskripte geschrieben, die das Testen erleichtern sollen (z.B. indem bestimmte Informationen angezeigt werden). Für diese Skripte existiert ein zentrales Debug-Menü, in dem sie ein- und ausgeschaltet werden können.

4.1 Verwenden des Debug-Menüs

Das Debug-Menü wird vollständig über die Schultertasten der beiden Controller gesteuert. Es erscheint als Text-Overlay über dem virtuellen Monitor. Das Drücken einer der beiden Schultertasten schaltet das Menü ein, ehe es nach zwei Sekunden wieder verschwindet.

Die linke Schultertaste wird dabei verwendet, um das Menü aufzurufen und durch die einzelnen Optionen durchzublättern. Ist das Menü aktuell ausgeblendet, wird es über die linke Schultertaste eingeblendet. Ist es eingeblendet, schaltet die linke Schultertaste zur nächsten Option weiter. Nachdem man am Ende angelangt ist, blättert man wieder von vorne los.

Mit der rechten Schultertaste wird die aktuell ausgewählte Option umgeschaltet.

4.2 Vorhandene Debug-Optionen

Von den Debug-Optionen schaltet die meisten lediglich einige Objekte an- und aus, andere können allerdings die Steuerung mit den Thumbsticks an andere Objekte als den virtuellen Monitor weitergeben, sodass sich dieser nicht mehr verschoben werden kann. Es kann immer nur eine Steuerung gleichzeitig erfolgen (wobei die Rangfolge bei mehreren aktiven Steuerungen wohldefiniert ist, siehe 4.3; das Verschieben des Monitors hat dabei die geringste Priorität), weshalb darauf geachtet werden sollte, dass Debug-Optionen zum Verschieben von Objekten wieder deaktiviert werden, sobald sie nicht mehr benötigt werden.

Gegenwärtig sind die folgenden 17 Debug-Optionen vorhanden:

4.2.1 Show Camera

Diese Option zeigt den Kamerastream der Webcam an. Auf diesem werden die aktuell erkannten Objekte als Rahmen farbig markiert. Gleiches gilt für die Kalibrierungspunkte.

Außerdem werden im Raum farbige Rahmen um die erkannten Objekte gelegt. Damit man

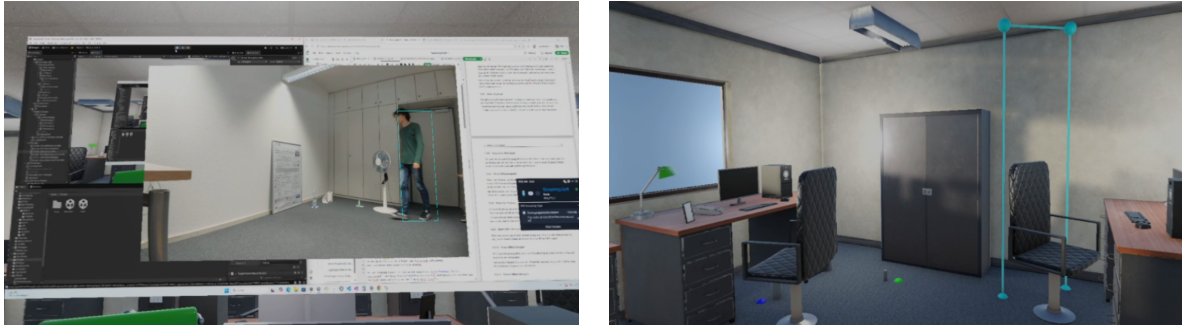


Abbildung 11: Links: Das Kamerabild mit einer erkannten Person. Rechts: Die Projektion des gefundenen Objektes in den Raum. Dort, wo jetzt der blaue Rahmen ist, befände sich eigentlich ein Passthrough.

diese vernünftig sehen kann, sollte zugleich die Option `Hide Detection Passthroughs` aktiviert sein.

4.2.2 Play Video

Ersetzt den Kamerastream der Webcam durch ein aufgezeichnetes Video. Dieses ist auch über `Show Camera` sichtbar (und wurde für Abb. 11) verwendet). Die Objekterkennung nutzt ebenfalls dieses Video.

4.2.3 Hide Detection Passthroughs

Hierüber werden die projizierten Passthrough-Cylinder der Objekterkennung entfernt. Dadurch werden insbesondere die Rahmen von `Show Camera` besser sichtbar.

Idealerweise sollte diese Option schon beim Start der Anwendung aktiviert werden, denn die eigentlichen Passthroughs werden nicht vollständig entfernt. Sie bleiben als helle Stellen im Hintergrund, was sich dadurch bemerkbar macht, dass das Hintergrund-Passthrough beim Fokuseffekt an diesen Stellen nicht dunkler wird. Dies stört aber nicht sonderlich, und da es sich hierbei nur um eine Debug-Option handelt, wurde dieser „Fehler“ nicht weiter behoben.

4.2.4 Show Environment

Zeigt eine vollständige VR-Umgebung an, sodass das Passthrough nicht mehr sichtbar ist. Dies wurde für die meisten Screenshots in dieser Dokumentation verwendet, z.B. in Abb. 11.

Diese Option wurde eingebaut, um Screenshots und Videos der Anwendungen anzufertigen, da die VR-Brille es in der Standardkonfiguration nicht erlaubt, das Passthrough

aufzunehmen, und stattdessen eine schwarze Fläche anzeigt.

Bei der Umgebung handelt es sich um ein kostenloses Asset⁴ aus dem „Unity Asset Store“, dessen Materialien von uns für das Rendern mit der modernen Universal Render Pipeline (URP) angepasst werden mussten.

4.2.5 Move Keyboard

Ermöglicht es, das Passthrough für die Tastatur zu verschieben. Dieses lässt sich dann mit dem linken Thumbstick in horizontaler Richtung verschieben und mit dem rechten Thumbstick nach oben und unten verschieben sowie um die vertikale Achse drehen.

4.2.6 Alternative Movement

Aktiviert eine alternative Steuerung für den virtuellen Monitor. Anders als bisher lässt sich der Monitor nun mit dem linken Thumbstick beliebig in horizontaler Richtung bewegen, zudem dreht er sich nicht mehr in Richtung Mittelpunkt.

4.2.7 Record Measurements

Startet die Aufnahme von Ablenkungen, Pupillengröße und Augenbewegungen. Diese werden als CSV-Dateien im Projektordner in jeweils einer Datei, die mit einem Zeitstempel versehen ist, gespeichert. Auf dem PC im VR-Labor befindet sich der Ordner und folgendem Pfad:

```
C:\Users\Studis\AppData\LocalLow\DefaultCompany\FocusGuard\Measurements
```

Die Werte werden vom Programm in die Dateien gestreamt, sodass es möglich ist, den Inhalt der zu prüfen, noch während die Aufnahmen laufen.

4.2.8 Show Eye Tracking

Zeigt zwei Kugeln an, die in die Blickrichtung des Benutzers zeigen. Dabei stellt die grüne Kugel das linke und die rote Kugel das rechte Auge (siehe Abb. 12). Falls das Eye-Tracking auf der Brille vernünftig kalibriert ist, sollten beide Kugeln an derselben Position liegen.

Die beiden Kugeln werden immer im Abstand von 2 m vor dem Benutzer platziert. Daher kann es sein, dass sie vom virtuellen Bildschirm verdeckt werden.

⁴„Office environment“ von Marpa Studio [9], mittlerweile als veraltet aus dem Asset Store entfernt.

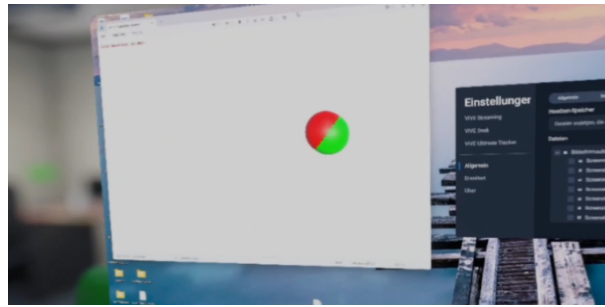


Abbildung 12: Eye-Tracking-Kugeln

4.2.9 Show Effect Strength

Wird diese Option angeschaltet, erscheint knapp unter dem virtuellen Monitor eine Anzeige, die die aktuelle Stärke des Fokuseffekts (von 0% bis 100%) angibt.

4.2.10 Freeze Effect Strength

Wird diese Option ausgewählt, bleibt der Fokuseffekt bei der aktuellen Stärke stehen und wird nicht mehr aktualisiert.

Insbesondere bedeutet dies, dass der Fokuseffekt dauerhaft ausgeschaltet bleibt, wenn diese Option bei Programmstart aktiviert ist.

4.2.11 Control Effect Strength

Diese Option sorgt dafür, dass man die aktuelle Stärke des Fokuseffektes manuell einstellen kann, indem man den rechten Controller nach oben und unten kippt (siehe Abb. 13).

Idealerweise nutzt man diese Option, indem man zunächst die beiden anderen Optionen „Show Effect Strength“ und „Freeze Effect Strength“ anschaltet, danach diese Option aktiviert, den Controller so kippt, dass die gewünschte Stärke erreicht ist, und anschließend diese Option wieder deaktiviert.

4.2.12 Move Webcam, Move Calibration Objects

Diese Optionen dienen während der Kalibrierung der Kamera (siehe 2.3) dazu, die Kalibrierungsobjekte und die Webcam zu verschieben. Damit diese Objekte sichtbar sind, muss die Option „Show Camera“ aktiviert werden.

Die Objekte können mit dem linken Thumbstick in horizontaler Richtung verschoben werden und mit dem rechten Thumbstick in vertikaler Richtung verschoben sowie um die vertikale Achse rotiert werden. Dabei ist zu beachten, dass die Steuerung weiterhin

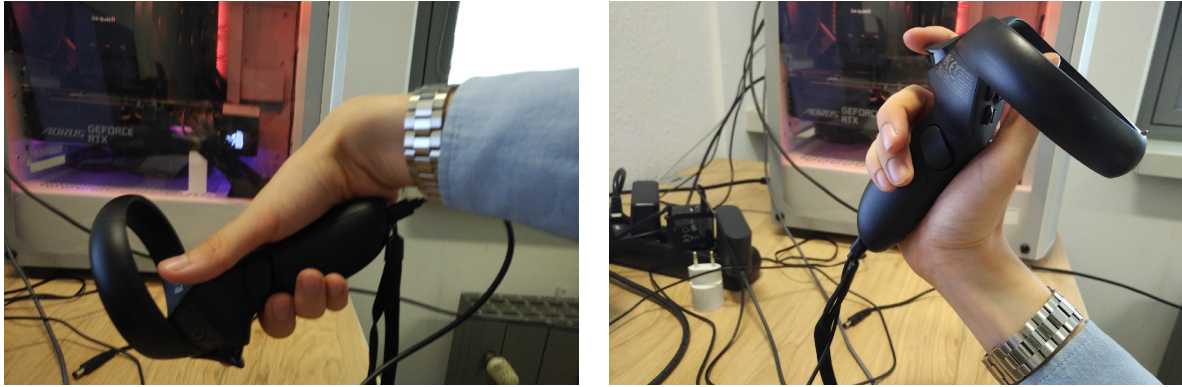


Abbildung 13: Links: Controllerposition für eine Effektstärke von 0%. Rechts: Position für 100%.

davon ausgeht, dass der Benutzer in Richtung des Bildschirms blickt, wodurch sich die Steuerung unintuitiv anfühlt, da der Benutzer ja in Richtung der Objekte blicken wird. Da dies aber nur während der initialen Kalibrierung gemacht werden muss, ist dies kein schwerwiegendes Problem.

Die Rotationsachse des Webcamobjektes befindet sich an deren Vorderseite, an der sich auch die eigentliche Kamera befindet.

4.2.13 Move Towel Roll, Move Water Bottle, Move Disinfectant

Diese Optionen aktiviert die Verschiebung drei Kalibrierungspunkte auf dem Webcam-Bild, dass zunächst mit „Show Camera“ angezeigt werden muss. Die Punkte können mit dem linken Thumbstick in horizontale und vertikale Richtung verschoben werden.

4.2.14 Optimize Webcam Rotation

Wird diese Option eingeschaltet, wird versucht, das Webcamobjekt so zu rotieren, dass eine Projektion der drei Kalibrierungspunkte möglichst nah an den drei Kalibrierungsobjekten liegt. Sie sollte also nur eingeschaltet werden, nachdem alle Objekte an ihre Position verschoben wurden.

Die Optimierung wird jedes Mal erneut ausgeführt, wenn die Option angeschaltet wird, nachdem sie vorher ausgeschaltet wurde.

4.3 Hinzufügen neuer Debug-Optionen

Alle Debug-Optionen sind als einfache Unity-Skripte realisiert. Ein zentraler Debug-Controller sorgt dafür, dass diese ein- und ausgeschaltet werden. Dieser befindet sich in der Hierarchie unter „Focus > Scripts > Script: DebugController“. Dort werden alle

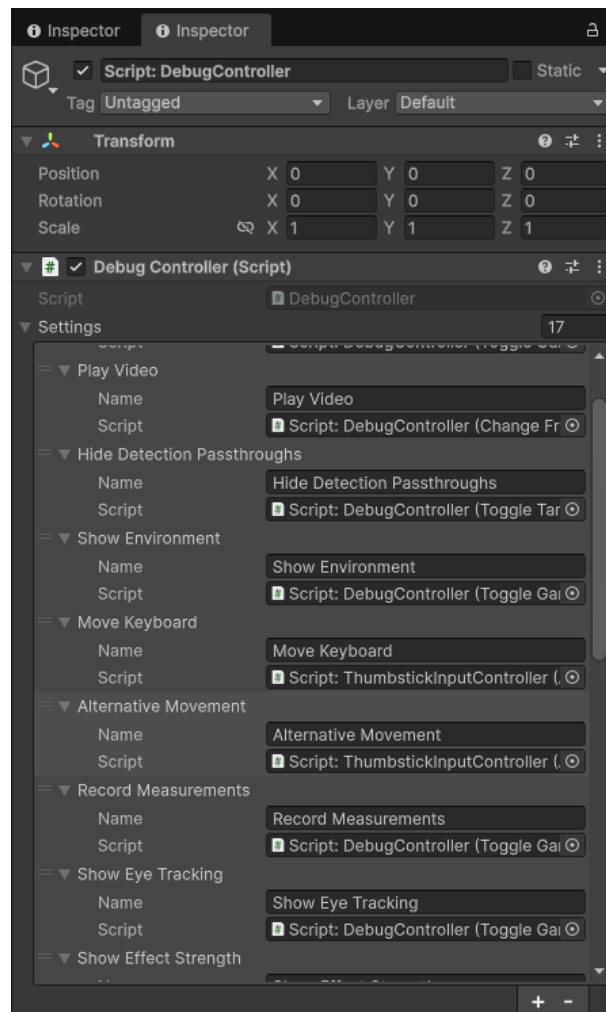


Abbildung 14: Debug-Controller mit Liste aller Debug-Optionen im Inspektor

Debug-Skripte namentlich in einer Liste registriert und erscheinen danach automatisch im Menü (siehe Abb. 14).

Die meisten Debug-Skripte selbst befinden sich ebenfalls am Debug-Controller. Die einzige Ausnahme bilden alle Debug-Skripte, die das Verschieben von Objekten betreffen. Da diese Zugriff auf die Thumbsticks benötigen, müssen sie auch beim Thumbstick-Input-Controller registriert werden, der sich in der Hierarchie unter „Focus > Scripts > Script: ThumbstickInputController“ befindet. An diesem GameObject sind daher diese zusätzlichen Skripte zu finden.

4.4 Standardwerte für Debug-Optionen

Da es sich bei den Debug-Optionen nur um Skripte handelt, die ein- und ausgeschaltet werden, ist es auch möglich, diese direkt im Unity-Inspektor ein- und auszuschalten. Der Debug-Controller berücksichtigt dabei diese Standardwerte. Es ist also möglich, bestimmte Voreinstellungen vorzunehmen, wenn man bestimmte Funktionen testen möchte und

sich dabei nicht immer durch das Debug-Menü klicken will.

Möchte man z.B. den Fokuseffekt ausprobieren, bietet es sich an, folgende Optionen anzuschalten:

- Show Effect Strength
- Freeze Effect Strength
- Control Effect Strength

Um die Objekterkennung auszutesten, sind diese Optionen sinnvoll:

- Show Camera
- Play Video
- Hide Detection Passthroughs, da diese die Rahmen im Raum verdecken.
- Freeze Effect Strength, damit man nicht vom Fokuseffekt abgelenkt wird.

Da sich die Debug-Skripte ja nur an zwei verschiedenen GameObjects befinden, lässt sich leicht kontrollieren, ob aktuell Debug-Optionen standardmäßig aktiviert sind. Um alle Debug-Optionen zu deaktivieren, dürfen beim Debug-Controller-GameObject nur die beiden Skripte „Debug Controller“ (bei dem die Debug-Optionen registriert sind) sowie „Debug Controller Input“ (das sich um die Steuerung des Debug-Menüs mit den Schultertasten kümmert) aktiviert sein. Beim Thumbstick-Input-Controller-GameObject dürfen lediglich die Skripte „Thumbstick Input Controller“ (für die Registrierung der Skripte, die Zugriff auf die Thumbsticks benötigen) sowie das oberste „Orbit Transform Controller“-Skript (das für das Verschieben des Bildschirms verantwortlich ist) aktiviert sein.

5 Systemarchitektur und Implementierung

Die Implementierung von FocusGuard besteht aus mehreren miteinander verbundenen Komponenten, die in einzelne Module aufgeteilt sind. Dieser Abschnitt beschreibt die wesentlichen Module und deren Zusammenhänge.

Die Hauptfunktionalität der Anwendung lässt sich in drei Teile untergliedern, die im wesentlichen voneinander unabhängig sind: Zum einen muss basierend auf Messungen der aktuelle Ablenkungs- bzw. Fokuszustand des Benutzers erkannt werden, sodass der Fokuseffekt angewandt werden kann. Hierfür sind die Module **Measurements** und **Effects** zuständig. Eine weitere wichtige Funktion besteht in der Objekterkennung und der damit verbundenen Einblendung von Objekten durch partielle Passthroughs. Dies ist die Aufgabe der Module **Detection** und **Webcam**.

Der dritte Teil besteht aus dem virtuellen Bildschirm und der Möglichkeit, diesen zu verschieben. Die Verschiebung ist im Modul „Transform“ implementiert und vollständig vom virtuellen Bildschirm unabhängig ist. Letzterer ist in einer einzigen Klasse im Namespace „UISwap“ implementiert, was das Standardnamespace für Klassen ist, die keinem größeren Modul zugeordnet werden können.

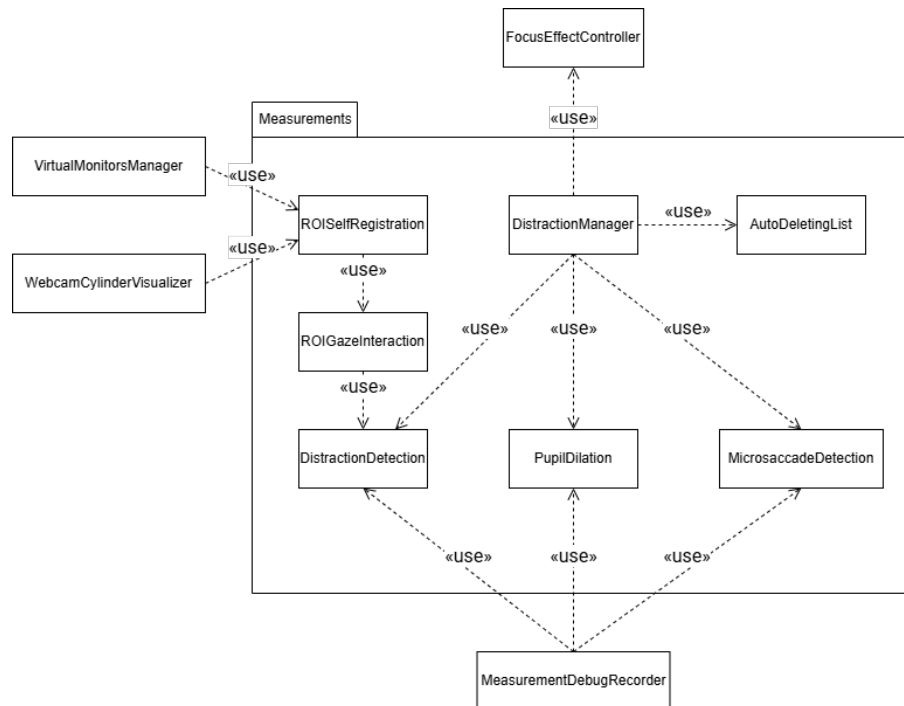


Abbildung 15: Paketdiagramm zur Measurements-Architektur

5.1 Modul „Measurements“

In diesem Modul geht es um die Erfassung der verschiedenen Eye-Tracking-Daten, sowie die Weitergabe dieser an andere Komponenten des FocusGuard-Systems. In FocusGuard benötigen drei Systemabschnitte Zugang zu Daten über den aktuellen Augenzustand des Nutzers. Dabei handelt es sich um die Systeme der Ablenkungserkennung sowie die Fokuserkennung, welche in die Teile Mikrosaccadenerkennung und Pupillenmessung unterteilt wird. Die Ablenkungserkennung und die Mikrosaccadenerkennung verwenden Daten über die Blickrichtung, während die Pupillenmessung nur Daten verarbeitet, welche die aktuelle Pupillengröße des Benutzers beinhalten.

Die Eye-Tracking-Daten werden von den entsprechenden Komponenten gesammelt und bewertet. Anschließend wird von der Komponente Distraction Manager entschieden ob der Fokus Effekt an- oder ausgeschaltet werden soll. Um ihn anzuschalten und damit eine Ausblendung zu starten benötigt es eine Schwellenüberschreitung der Ablenkungen. Um den Fokus Effekt wieder auszuschalten muss der Benutzer für eine gewisse Zeit auf die Region of Interest schauen und es benötigt eine Bestätigung von den beiden Fokuserkennungen. Da diese jedoch unverlässlich sind wurden sie aktuell deaktiviert.

5.1.1 Uniformes Listensystem

Die von der Brille erfassten Eye Tracking Daten werden mit Hilfe der Vive API für die Verwendung in Unity freigegeben. Dabei werden in jedem Frame die aktuellen Augenrich-

tungen und Pupillengrößen von beiden Augen erfasst und abgespeichert. Um die Daten für die verschiedenen Systeme sinnvoll speichern und später auswerten zu können wurde eine neue Variante der Datenstruktur Liste erstellt, die Auto-Deleting List. Diese verhielt sich so ähnlich wie eine normale C# Liste, mit der Änderung, dass sie bei einem Durchlauf automatisch Einträge, die eine gewisse Bedingung erfüllen, aus der Liste entfernt. Bei der Erstellung einer neuen Instanz einer Auto-Deleting List wird ihr eine Funktion übergeben, welche den Rückgabewert Boolean hat. Wenn die Liste später durchlaufen wird, so wird das aktuelle Element zunächst mit dieser Funktion überprüft und sollte das Element die Funktion erfüllen, so wird es gelöscht.

Diese neue Datenstruktur wurde verwendet, um die veralteten Augenmessungen zu entfernen, bevor sie für die unterschiedlichen Systeme verwendet wurden. Es wurden drei Listen erstellt, die jeweils die Ablenkungen, die Mikrosaccaden und die Pupillengrößen beinhalteten. Diese erhielten regelmäßig einen neuen Eintrag, der den Zeitpunkt des Frames enthielt. Die Funktionen, die den Listen bei Erzeugung übergeben wurden, sortierten die veralteten Einträge heraus, während die Listen in ihren Komponenten verwendet wurden.

5.1.2 Ablenkungserkennung

In dieser Komponente wurden Ablenkungen in der Liste gespeichert. Jede Ablenkung war ein Objekt, welches über einen Anfangs- und Endzeitpunkt verfügte sowie über einen Wert, der aussagte, ob die Ablenkung vorbei war. Es wurden die Richtungsdaten des Eye Trackings genutzt, um jedes Mal, wenn der Benutzer von der Region of Interest wegschaute, ein neues Ablenkungsobjekt zu erzeugen. Dieses erhielt beim Wegschauen einen Startzeitpunkt gespeichert und einen Endzeitpunkt, sobald der Nutzer wieder zurück schaute. Zusätzlich wurde von der Komponente Distraction Manager in jedem Frame die Liste der Ablenkungen überprüft, um zu sehen ob die vorab gesetzten Schranken überschritten wurden. Wenn eine zu lange oder genügen kurze Ablenkungen in einem Zeitfenster erkannt wurden, wurde der ausgewählte Fokus Effekt ausgelöst und somit mit der Ausblendung der Umgebung begonnen.

5.1.3 Mikrosaccadenerkennung

Für die Mikrosaccadenerkennung wurden die Richtungsdaten der Augen verwendet. Diese wurden in jedem Frame in eine eigene Liste gespeichert und später für die Auswertung zu verwenden. Die Auswertung der Daten beruhte auf einem Algorithmus von Engbert und Kliegl fand im zweidimensionalen Geschwindigkeitsraum statt [4]. Dabei wurden die aufgenommenen Augenbewegungen in geglättete Geschwindigkeiten umgewandelt, von welchen dann, zur Reduktion von Messfehlern, der Median berechnet wurde. Die resultie-

renden Geschwindigkeiten wurden anschließend mit Schwellenwerten verglichen und als Mikrosaccaden gewertet, wenn sie diese überschritten.

Falls Mikrosaccaden erkannt werden, wird eine Bestätigung an den Distraction Manager weitergeleitet, dass dieser die Ausblendung mit dem Fokus Effekt auflösen kann.

Jedoch waren die Messungenauigkeiten des Eye Tracking so groß, dass selbst mit der Mittelwertbestimmung, die gesuchten Mikrosaccaden nicht verlässlich von dem Rauschen der Messungen zu unterscheiden waren. Hierfür reicht die Genauigkeit der Brille (Ausgabe mit 120 Hz, Genauigkeit von 0.5° bis 1.1° [3]) nicht aus. Nach externer Auswertung eines Testdatensatzes, mit Hilfe des Debug Recorders, und Absprache mit den Betreuern, wurde die Mikrosaccadenerkennung für Fokuserkennung deaktiviert.

5.1.4 Pupillengröße

Bei dieser Komponente werden in jedem Frame die Größe der beiden Pupillen des Benutzers gemessen und in eine eigene Liste gespeichert. Diese können dann in jedem Frame mit vorher eingestellten Vergleichswerten verglichen werden um zu entscheiden, ob eine fokusbasierte Vergrößerung der Pupillen vorliegt. Falls dies der Fall ist, wird eine Bestätigung an den Distraction Manager weitergeleitet, dass dieser die Ausblendung mit dem Fokuseffekt auflösen kann.

Jedoch sind die Unterschiede der Pupillengrößen, die durch unterschiedliche Helligkeiten entstehen, deutlich größer als die Weitung bei einer Fokussierung, was zu einer Verwerfung der Pupillengröße zur Fokuserkennung führte.

5.1.5 Debug Recorder

Während die bisher genannten Komponenten die Eye-Tracking-Daten für Ablenkungs- und Fokuserkennung verwendeten, konnte über das Debug-Menü aktiviert werden, dass die erfassten Augendaten für externe Auswertungen gespeichert werden. Dazu wurde ein Debug Recorder erstellt, welcher in einem dafür ausgewählten Ordner auf dem PC eine CSV-Datei erzeugte und in diese alle Augendaten von jedem Frame abspeicherte. Um zu entscheiden, welche Daten in die CSV Datei gespeichert werden sollen wurde eine erweiterbare Liste mit allen Spalten erstellt, die in der Tabelle erscheinen sollten.

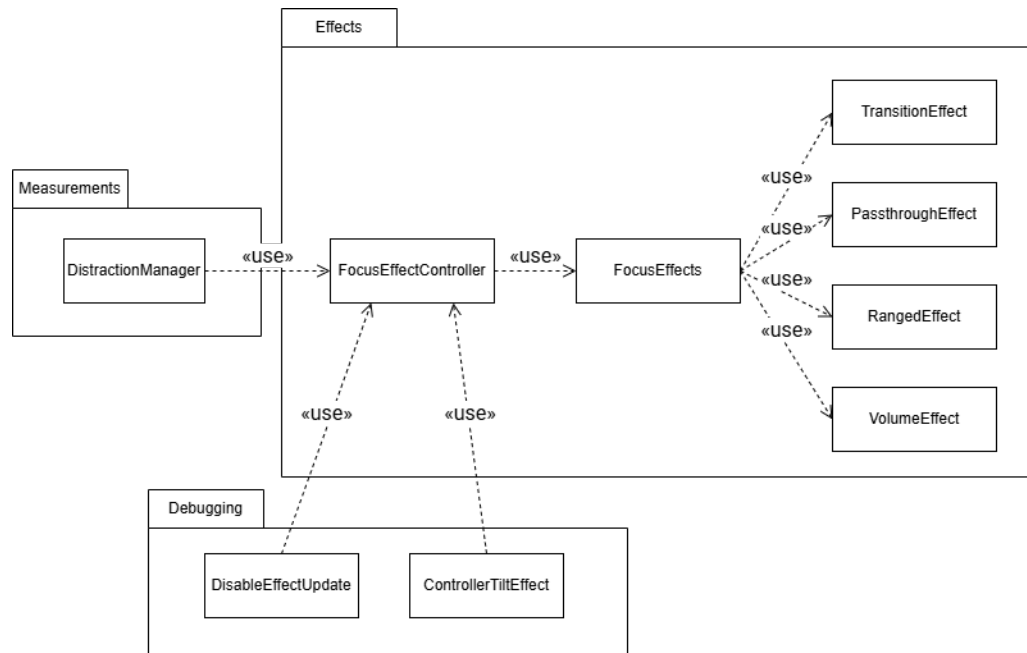


Abbildung 16: Paketdiagramm des Effekt-Moduls und dessen Verwendung

5.2 Modul „Effects“

Dieses Modul ist für die Darstellung des Fokuseffektes verantwortlich. Ein Fokuseffekt kann jede beliebige Funktion sein, die einen Parameter **strength** besitzt, der einen Wert zwischen 0 und 1 annehmen kann. Hierbei steht 0 dafür, dass der Effekt vollständig ausgeschaltet ist, während der Effekt bei einem Wert von 1 vollständig angeschaltet ist. Inspiriert durch das „Milgram reality-virtuality continuum“ [8] lassen sich so auch Zwischenzustände auf dem Weg zur vollständigen VR-Umgebung erreichen.

Die zentrale Klasse des Moduls ist der **FocusEffectController**. Im Unity-Inspektor befindet er sich unter „Focus > Scripts > Script: FocusEffectController“. Er enthält die Definition für den Fokuseffekt, den die Anwendung tatsächlich verwendet (vgl. 3.2) und verwaltet die aktuelle Effektstärke. Dazu stellt er eine Eigenschaft namens **EffectStrength** bereit, über die der **DistractionManager** den Zielwert für den Fokuseffekt einstellt (aktuell wird nur 0 oder 1 verwendet, theoretisch wäre es aber auch möglich, jeden beliebigen Wert in diesem Wertebereich als Zielwert zu verwenden). Die Transition zu diesem Zielwert wird dabei durch den **FocusEffectController** automatisch gesteuert.

Zudem besitzt der **FocusEffectController** noch Schnittstellen, um die Debug-Optionen „Freeze Effect“ und „Control Effect Strength“ zu ermöglichen, die die Zielstärke des Effektes abweichend von der Ablenkungserkennung setzen.

Die Funktionen für die Fokuseffekte selbst werden in anderen Klassen definiert. Ursprünglich wurde hierfür eine Klassenhierarchie erstellt, die auf einer Schnittstelle namens **IFocusEffect** basierte und die notwendige Methode bereitstellte. Mittlerweile wird dafür

jedoch ein Delegattyp verwendet, wodurch die Definition von neuen Effekten zu Einzeilern innerhalb einer statischen Fabrikmethode in der Klasse `FocusEffects` wurde:

```

/// Setzt die Hintergrundfarbe der Hauptkamera.
/// (Berechnung der Farbe leicht vereinfacht).
public static FocusEffect BackgroundColor(Color color)
{
    return strength => MainCamera.backgroundColor = color * strength;
}

```

Eigene Effektklassen existieren nur noch für komplexere Effekte. Mehrere Effektfunktionen können über einen Union-Effekt miteinander kombiniert werden. Dies führt zu einer besonders eleganten Definition des gesamten Fokuseffekts der Anwendung:

```

transitionEffect = Union(
    Passthrough().Invert(),
    Vignette(),
    Blur(),
    BackgroundColor(Color.white).AddRange(0.5f, 1.0f, null)
).AddSmoothstepTransition(this, duration: 10.0f);

```

Viele für unser Projekt interessante Fokuseffekte (z.B. der Blur-Effekt oder Vignette) sind als Post-Processing-Effekt in Unity realisiert. Um einen Post-Processing-Effekt anwenden zu können, ist ein Zugriff auf das bereits gerenderte Bild erforderlich. Dies führte wieder zu einem Problem mit der VR-Brille, da wir ja keinen Zugriff auf das Passthrough-Bild erhalten (stattdessen rendern wir in Unity ein Bild mit transparentem Hintergrund, unter das die Brille dann das Passthrough legt). Dadurch waren wir in der Wahl der Effekte stark eingeschränkt.

Sinnvoll verwendbar ist daher nur der `PassthroughEffect`, der die Stärke des Passthroughs anpasst, wodurch das Passthrough zunehmend dunkler wird, sowie ein einfacher `BackgroundColorEffect`, der die Transparenz der Hintergrundfarbe der Unity-Kamera anpasst. Der Blur-Effekt ist beim Passthrough nicht sichtbar, und die Vignette wird erst sichtbar, sobald der `BackgroundColorEffect` einsetzt, und hat dabei dieselbe Transparenz wie die Hintergrundfarbe.

Beim Passthrough-Effekt steht ein Wert von 1 dafür, dass das Passthrough vollständig aktiviert ist, und ein Wert von 0 dafür, dass es vollständig deaktiviert ist. Dies ergibt intuitiv Sinn, allerdings soll das Passthrough der Anwendung ja genau gegenteilig aktiviert werden, d.h. bei einem Fokuseffekt von 1 deaktiviert sein. Daher wird die Stärke dieses Effektes invertiert.

Post-Processing-Effekte werden bei der Unity Universal Render Pipeline (URP) einge-

stellt, indem die Einstellungen einer „Volume“-Komponente angepasst werden. Daher existiert eine Klasse `VolumeEffect`, die Zugriff auf dieses Volume ermöglicht. Diese wird vom Blur-Effekt und der Vignette verwendet.

Über den `RangedEffect` ist es möglich, einen Fokuseffekt erst später starten zu lassen, wie es aktuell in der Anwendung mit dem weißen `BackgroundColorEffect` geschieht, der erst ab einem Wert von 0,5 einsetzt. Hierbei gibt man an, wann ein Effekt beginnt, das Maximum erreicht, und ggf. auch wieder endet.

Der `TransitionEffect` ist dafür verantwortlich, den vollständigen Effekt erst mit einer Verzögerung anzuwenden und eine angenehme Transition zur Zieleffektstärke zu ermöglichen. Dabei kann jede beliebige Funktion $f : [0, 1] \rightarrow [0, 1]$ für die Transition verwendet werden. Eine lineare Funktion $f(x) = x$ ist dabei ausreichend, für die Anwendung haben wir allerdings (hauptsächlich, um diese Möglichkeit zu demonstrieren) die Smoothstep-Funktion $f(x) = x \cdot x(3 - 2x)$ verwendet, die ein Beispiel für eine Sigmoidfunktion darstellt.

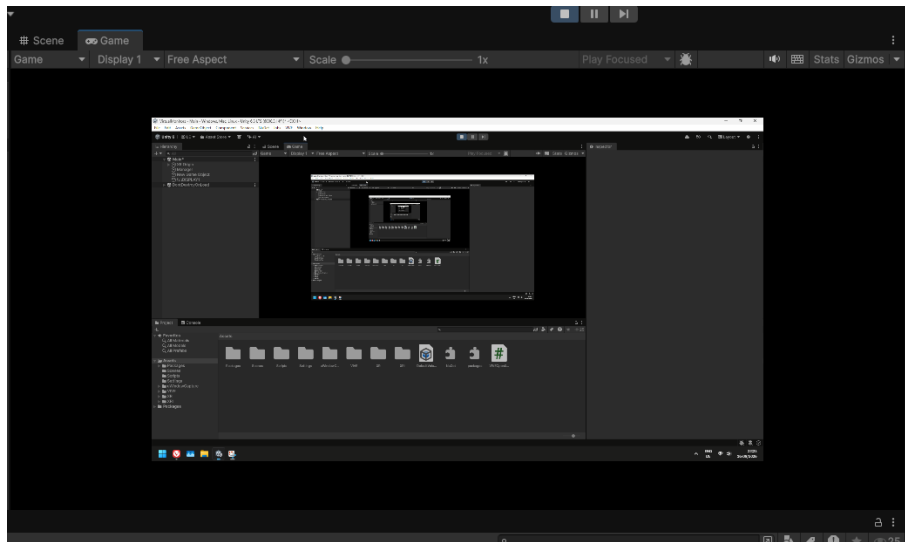


Abbildung 17: Das ursprüngliche Aussehen von Virtual-Monitors in Unity

5.3 Implementierung des virtuellen Bildschirmes

5.3.1 Absichten

Ziel dieses Abschnitts der Systemarchitektur war es, einen funktionierenden Bildschirm für die Verwendung in der VR-Brille zu erzeugen. Viele der kommerziell verfügbaren VR-Brillen verfügen über eine eingebaute App, die es der Brille ermöglicht, sich mit einem PC zu verbinden und den Desktop auf ein UI-Fenster in der Brille zu projizieren. Auch bei der HTC Vive Focus Vision, die die VR-Brille ist, die in diesem Projekt verwendet wurde, ist solch eine Funktion in der App „Virtual Desktop“ vorhanden. Jedoch ist dies eine in sich abgeschlossene App, welche ihre Features, anders als die für Developer frei zugänglichen Funktionen wie das Passthrough oder Eye Tracking, nicht für andere Apps zur Verfügung stellt. Somit war die einzige Option für die Erzeugung eines virtuellen Bildschirms in dem FocusGuard-System, die Funktion separat zu implementieren.

5.3.2 Anpassung des Virtual-Monitors-Projektes

Um die Implementierung eines virtuellen Bildschirms für das Projekt in einem realistisch umsetzbaren Rahmen zu halten, wurde als Kernstück der Systemarchitektur ein bereits existierendes C#-Projekt verwendet. Nach einer Suche und Auswahl entschied sich das Team für das Projekt „Virtual-Monitors“, welches in einem Github Repository von StevenRice99 veröffentlicht wurde⁵.

Virtual-Monitors verwendet das Windows Display API Packet, um alle vorhandenen Desktops zu erfassen und die notwendigen Daten für die Erstellung der Bildschirme zu

⁵<https://github.com/StevenRice99/Virtual-Monitors/>, MIT-lizenziert

erhalten. Es verwendet die aufgenommenen Daten, um für jeden Desktop ein neues UI-Element in Unity zu erstellen und den aufgenommenen Bildschirm auf sie zu projizieren. Zusätzlich erfasst es die Mausposition und zeigt diese auf dem entsprechenden Desktop an.

Somit war es möglich, einen Desktop auf ein Unity-UI-Element zu projizieren. Leider war das genutzte C#-Projekt nur für Unity Projekte ausgelegt, welche auf einem Windows-PC laufen. Da das Programm die Desktopsdaten vom Computer auslesen muss und dies mit Hilfe einer Windows-API zustande bringt, war es nicht möglich, die Architektur auf der vorhandenen VR-Brille zu betreiben, welche ein Android-Betriebssystem hat. Jedoch hatte das Virtual-Monitors Projekt auch Probleme, die Desktopdaten vom PC an die Brille zu schicken, wenn das Programm auf dem PC lief.

Damit der virtuelle Bildschirm in die VR-Brille projiziert werden kann und dort sichtbar ist, wurde eine Verbindung zwischen PC und Brille erzeugt. Dazu wurden die zwei Programme ViveHub und SteamVR genutzt. ViveHub ist eine Anwendung, die von den Herstellern der Brille zur Verfügung gestellt wird und eine Verbindung zwischen dem Computer und der Brille ermöglicht. Dies geht sowohl über eine WLAN-Verbindung als auch über Kabel. Bei der Verbindung über WLAN wurde der PC nicht von der Brille erkannt, weshalb die Kabelverbindung genutzt wurde. Nachdem eine Verbindung aufgebaut wurde, musste das Programm auf dem PC gestartet werden und die Bildschirmdaten über das Kabel auf die Brille geschickt werden. Dazu wurde SteamVR genutzt. SteamVR ist in der Lage, Programme von VR-Brillen auf dem PC zu starten und auf der angeschlossenen Brille zu bedienen.

In der aktuellen Version des OpenXR SDK von VIVE existiert ein Bug, der zu einem Absturz von Unity führt, wenn die Anwendung über SteamVR läuft und gleichzeitig Eye Tracking und Passthrough aktiviert sind. Hierfür haben wir einen Hotfix entwickelt und verwenden daher unseren eigenen Fork der OpenXR SDK⁶, anstatt die offizielle Version von VIVE zu verwenden.

Zusammen erlaubten ViveHub und SteamVR, die Desktopaufnahme auf dem PC zu starten und auf das dafür erstellte UI-Element in der Brille zu projizieren. Übrig blieben zwei ungelöste Probleme. Das eine ist, dass aus ungeklärten Gründen und nach stark unterschiedlichen Zeiten SteamVR abstürzt und somit das Programm neu gestartet werden muss. Das andere ist die Tatsache, dass das Programm nicht auf einer alleinstehenden Brille ausführbar ist, da die Desktopaufnahme auf dem PC laufen muss.

⁶<https://github.com/Klock-Moritz/VIVE-OpenXR-Unity>

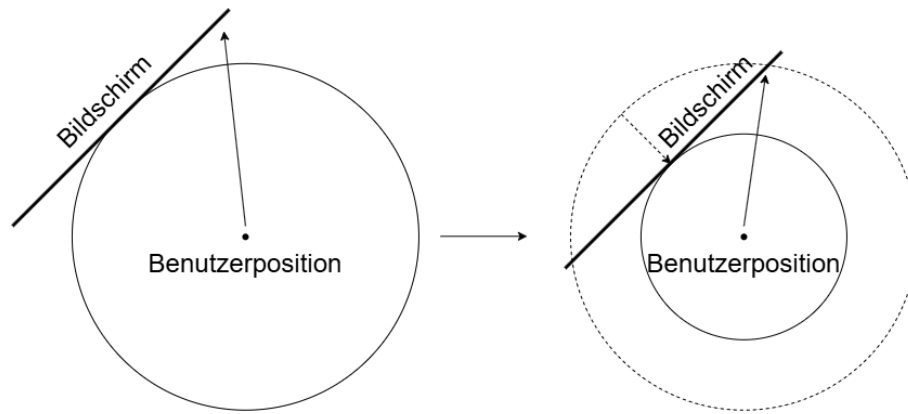


Abbildung 18: Bildschirmrotation im Verhältnis zu Benutzer

5.3.3 Bildschirmsteuerung

Nachdem der Desktop nun in der VR-Brille sichtbar war und funktionierte, wurden die Benutzerfunktionen ausgebaut. In dem ursprünglichen Projekt konnte eingestellt werden, in welchem Abstand zum Benutzer der Bildschirm erstellt werden sollte. Da dies jedoch nur wenig Spielraum für die Positionierung gab und keine Möglichkeit, die Position des Bildschirms nach dem Start zu bewegen, wurden verschiedene Verschiebungen eingebaut. Es wurde ermöglicht, den Bildschirm nach oben und unten sowie zu dem Benutzer hin und weg zu schieben. Zusätzlich wurde es möglich, den Bildschirm um den Benutzer zu rotieren und den Bildschirm um seine Längsachse zu drehen. Dies ermöglichte es, den Bildschirm so einzustellen, dass der Nutzer angenehm an ihm arbeiten konnte. Später wurde eine weitere Bewegung implementiert, bei der man den Bildschirm auf seiner Ebene seitwärts bewegen konnte, ohne ihn um den Benutzer zu schwenken.

Als letzter Teil wurde das Unity-Skript, welches die virtuellen Bildschirme erzeugte, erweitert, sodass die Bildschirme bei ihrer Erzeugung direkt zu einem Teil der Region of Interest wurden. Dazu wurde den Unity Objekten bei ihrer Erstellung ein weiterer Bestandteil hinzugefügt, welcher mit dem Gaze Interactor des Eye-Trackings interagieren konnte und somit erkannte, ob die Bildschirme gerade von dem Nutzer angeschaut wurden. Somit konnten die Skripte, welche für die Ablenkungs- und Fokuserkennung verantwortlich waren, auf den Bildschirmen funktionieren.

5.4 Objekterkennung

Damit relevante Objekte der realen Umgebung trotz der visuellen Effekte von FocusGuard sichtbar bleiben können, müssen diese zunächst zuverlässig erkannt werden. Diese Objekte werden im FocusGuard-Konzept als Critical Anchors bezeichnet. Im Rahmen der Implementierung liegt der Schwerpunkt insbesondere auf Personen als dynamischen Critical Anchors.

Die Objekterkennung hat dabei die Aufgabe, relevante Objekte innerhalb eines Kamerabildes zu identifizieren und ihre Position und Ausdehnung im Bild zu bestimmen. Dafür werden neben der erkannten Objektklasse auch die zugehörige Bounding Box und die Konfidenz der Erkennung bereitgestellt. Diese zweidimensionalen Detektionsinformationen bilden anschließend die Grundlage für die räumliche Integration, durch die das erkannte Objekt im dreidimensionalen Unity-Raum lokalisiert werden kann.

Das ursprüngliche FocusGuard-Konzept sieht für die Erfassung von Critical Anchors zwei unterschiedliche Ansätze vor. Stationäre Objekte sollen über native Scene APIs erfasst werden, während für dynamische Critical Anchors ein leichtgewichtiges YOLO-Modell vorgesehen ist, das innerhalb der Unity-Anwendung ausgeführt wird. Im Rahmen der Umsetzung auf der VIVE Focus Vision wurde zunächst geprüft, inwieweit sich die statischen Critical Anchors über die verfügbaren Funktionen zur räumlichen Umgebungserfassung abbilden lassen. Das im Paper vorgesehene Scene Understanding stand im verwendeten Standalone-Setup jedoch nicht in der benötigten Form zur Verfügung. Plane Detection kann zwar Flächen des zuvor eingerichteten Raums bereitstellen, aber nur außerhalb der App. Der im Paper vorgesehene Ansatz für stationäre Critical Anchors wurde daher im weiteren Projektverlauf nicht weiterverfolgt und der Schwerpunkt auf die Erkennung dynamischer Critical Anchors gelegt.

Für deren Umsetzung ergibt sich auf der VIVE Focus Vision eine zusätzliche technische Anforderung: Die Objekterkennung benötigt einzelne Kameraframes als auswertbare Bilddaten. Das Passthrough-System des Headsets ermöglicht zwar die Darstellung der realen Umgebung innerhalb der Mixed-Reality-Anwendung, über den im Projekt verwendeten OpenXR-Passthrough-Pfad konnte jedoch kein geeigneter Zugriff auf einzelne Kameraframes für die Verarbeitung durch das neuronale Netz etabliert werden.

Daher wurde die Bildquelle der Objekterkennung von der Passthrough-Darstellung getrennt. Als Bildquelle wird eine externe Webcam verwendet. Diese stellt ausschließlich die Bilddaten für die Objekterkennung bereit. Die Darstellung der realen Umgebung erfolgt weiterhin über das Passthrough-System der VIVE Focus Vision. Das Webcam-Bild selbst wird somit nicht als Passthrough in das Headset übertragen, sondern ausschließlich analysiert.

Eine alternative Architektur wäre grundsätzlich über das VIVE Wave SDK denkbar. Wa-

ve stellt gerätespezifische Kamera-Schnittstellen bereit und bietet damit einen möglichen Technologiepfad, über den die für YOLO benötigten Bilddaten direkt vom Headset bezogen werden könnten. FocusGuard basierte jedoch bereits auf VIVE OpenXR, sodass eine Wave-basierte Lösung einen Wechsel beziehungsweise eine zusätzliche Integration eines gerätespezifischen XR-Stacks erfordert hätte. Dieser Ansatz wurde im Rahmen des Projekts nicht auf der VIVE Focus Vision validiert. Daher kann nicht abschließend beurteilt werden, ob die benötigten Kameradaten darüber in geeigneter Form und mit ausreichender Performance verfügbar gewesen wären.

5.4.1 Architektur der Objekterkennung

Aus der Verwendung einer externen Kamera ergibt sich die Anforderung, die eigentliche Objekterkennung möglichst unabhängig von der konkreten Bildquelle zu implementieren. Die Detektionslogik soll nicht wissen müssen, ob ein Bild von einer Webcam, einer Videodatei oder zukünftig einer anderen Bildquelle stammt. Diese Trennung wird durch eine gemeinsame Abstraktion der verschiedenen Bildquellen umgesetzt.

Abbildung 19 zeigt den hierfür relevanten Teil der Paketstruktur.

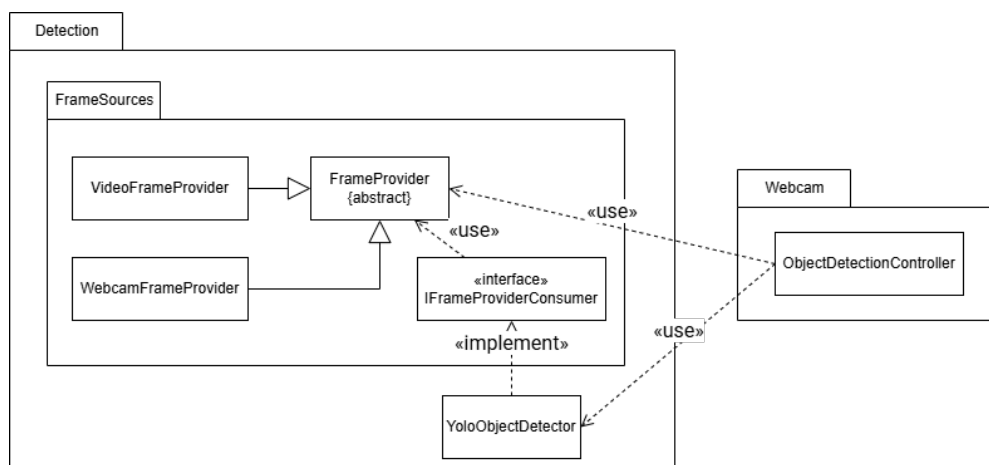


Abbildung 19: Paketstruktur der Objekterkennung mit abstrahierter Bildquelle.

Die zentrale Abstraktion bildet der **FrameProvider**. Er stellt der Objekterkennung eine einheitliche Schnittstelle für Bilddaten zur Verfügung. Konkrete Bildquellen kümmern sich selbst darum, wie der jeweilige Frame erzeugt beziehungsweise eingelesen wird. Im aktuellen System existieren hierfür insbesondere der **WebcamFrameProvider** für einen Live-Kamerastream und der **VideoFrameProvider** für Videomaterial.

Der **YoloObjectDetector** arbeitet mit dem bereitgestellten **FrameProvider** und ist dadurch nicht direkt an die externe Webcam gekoppelt. Die eigentliche Objekterkennung bleibt somit von der konkreten Herkunft des Kamerabildes getrennt.

5.4.2 Bildaufbereitung und YOLO-Inferenz

Für die eigentliche Objekterkennung wird YOLOv8n verwendet. Die Nano-Variante ist innerhalb der YOLOv8-Modellfamilie vergleichsweise kompakt und reduziert dadurch den Rechenaufwand der Inferenz. Dies ist insbesondere für FocusGuard relevant, da die Objekterkennung parallel zur interaktiven XR-Anwendung ausgeführt werden muss.

Das Modell liegt im ONNX-Format vor und wird mit Unity AI Inference direkt innerhalb der Unity-Anwendung ausgeführt. Das verwendete Inferenz-Backend kann im Unity-Inspector konfiguriert werden. Standardmäßig ist in der Implementierung `GPUCompute` vorgesehen, sodass die Berechnungen des neuronalen Netzes über die GPU ausgeführt werden können.

Die Verarbeitung eines Kameraframes folgt einer festen Pipeline. Zunächst stellt der konfigurierte `FrameProvider` einen Frame als Unity-Textur bereit. Der `YoloObjectDetector` prüft anschließend, ob die Bildquelle bereit ist, ein neuer Frame vorliegt und seit der letzten Inferenz das konfigurierte Zeitintervall vergangen ist. Erst dann wird eine neue Modellinferenz gestartet.

Das Inferenzintervall ist im Unity-Inspector einstellbar und beträgt standardmäßig 0,5 Sekunden. Die Framerate der Kamera und die Frequenz der Objekterkennung sind dadurch voneinander entkoppelt. Die Webcam kann beispielsweise 30 Bilder pro Sekunde bereitstellen, ohne dass für jedes einzelne Kamerabild eine vollständige YOLO-Inferenz durchgeführt werden muss. Mit der Standardkonfiguration wird höchstens ungefähr zweimal pro Sekunde eine neue Inferenz gestartet. Dadurch lässt sich ein Kompromiss zwischen Aktualität der Objekterkennung und benötigter Rechenleistung einstellen.

Auch die Webcam selbst ist konfigurierbar. Standardmäßig werden eine Auflösung von 1280×720 Pixeln und 30 Bilder pro Sekunde angefordert. Zusätzlich kann ein bestimmtes Kameragerät ausgewählt werden. Falls keine konkrete Webcam vorgegeben wird, kann auf ein verfügbares Kameragerät zurückgegriffen werden. Bei Auflösung und Bildrate handelt es sich um angeforderte Werte; die tatsächlich bereitgestellten Eigenschaften hängen von der verwendeten Kamera ab.

Eine besondere Rolle spielt die Aufbereitung des Kamerabildes für das neuronale Netz. Das verwendete YOLOv8n-Modell erwartet einen Eingabetensor mit einer festen Größe von $1 \times 3 \times 640 \times 640$. Die drei Kanäle repräsentieren die Farbinformationen des Bildes. Unabhängig von der tatsächlichen Auflösung des vom `FrameProvider` bereitgestellten Frames muss das Bild daher zunächst auf eine quadratische Eingabe mit 640×640 Pixeln überführt werden.

In der aktuellen Implementierung wird der Kameraframe mit `TextureConverter.ToTensor` direkt in den für das Modell vorgesehenen 640×640 großen Tensor überführt. Besitzt das

Ausgangsbild kein quadratisches Seitenverhältnis, werden die horizontale und vertikale Dimension dabei unterschiedlich skaliert. Bei der standardmäßig angeforderten Webcam-Auflösung von 1280×720 Pixeln wird beispielsweise ein Bild mit einem Seitenverhältnis von 16:9 auf das quadratische Seitenverhältnis von 1:1 des Modelleingangs abgebildet.

Dadurch besitzt ein Objekt während der Modellinferenz nicht mehr exakt dieselben geometrischen Proportionen wie im ursprünglichen Kamerabild. Diese Veränderung kann bereits die Objekterkennung selbst beeinflussen, da YOLO seine Merkmalsextraktion auf dem transformierten Bild durchführt. Formen, Konturen und Größenverhältnisse können durch die unterschiedliche Skalierung verändert werden. Dies kann sich sowohl auf die Klassifikationskonfidenz als auch auf die Genauigkeit der vorhergesagten Bounding Box auswirken.

Abbildung 20 stellt die aktuelle Bildaufbereitung einer möglichen seitenverhältniserhaltenden Variante gegenüber.

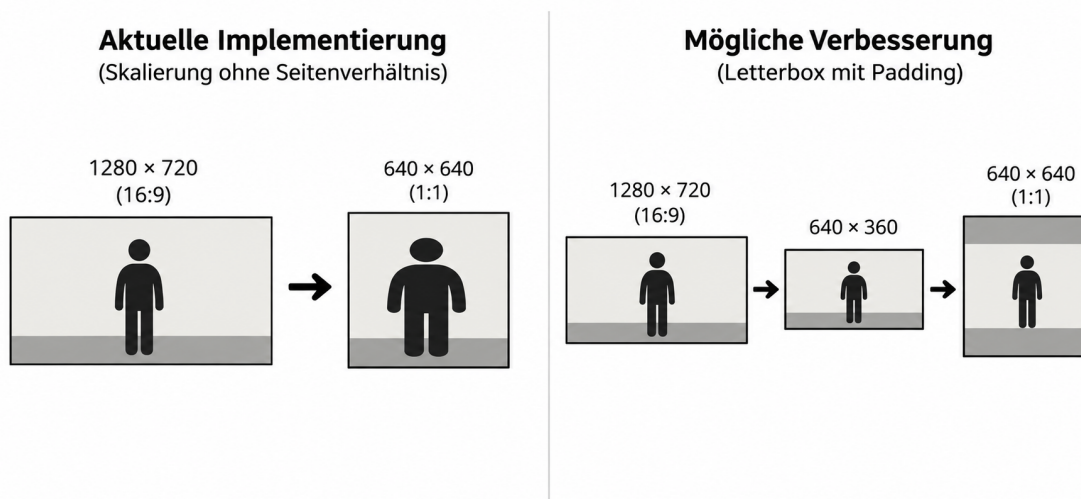


Abbildung 20: Aufbereitung eines beispielhaften 1280×720 -Kamerabildes für den quadratischen 640×640 -Eingang von YOLOv8n. Die aktuelle Skalierung verändert das Seitenverhältnis. Eine mögliche Alternative besteht in einer seitenverhältniserhaltenden Skalierung mit Padding.

Nach der Bildaufbereitung wird der erzeugte Tensor durch YOLOv8n verarbeitet. Die Ausgabe des Modells enthält zunächst eine große Anzahl potenzieller Objektdetektionen. Der verwendete YOLOv8n-Ausgang besitzt die Form $1 \times 84 \times 8400$. Die 8400 Einträge stellen mögliche Detektionskandidaten dar. Die ersten vier Werte eines Kandidaten beschreiben Position und Größe der Bounding Box, während die übrigen 80 Werte den vom verwendeten Modell unterstützten COCO-Objektklassen entsprechen.

Die vom Modell ausgegebenen Bounding-Box-Koordinaten beziehen sich zunächst auf die

Modellauflösung von 640×640 Pixeln. Nach der Inferenz werden diese Koordinaten wieder auf die tatsächliche Größe des aktuell verarbeiteten Frames zurückskaliert. Die Rückskalierung ist dabei nicht an eine feste Webcam-Auflösung gebunden, sondern berücksichtigt dynamisch die Breite und Höhe des jeweiligen Eingabeframes. Die horizontalen und vertikalen Skalierungsfaktoren ergeben sich aus

$$s_x = \frac{W_{\text{Frame}}}{640}, \quad s_y = \frac{H_{\text{Frame}}}{640}.$$

Die Koordinaten und Abmessungen der Bounding Box werden mit den entsprechenden Skalierungsfaktoren in das Koordinatensystem des ursprünglichen Frames übertragen. Bei einem Frame mit 1280×720 Pixeln ergeben sich beispielsweise die Faktoren $s_x = 2$ und $s_y = 1,125$. Bei einer anderen Framegröße werden entsprechend andere Skalierungsfaktoren verwendet. Die weitere Verarbeitung ist dadurch nicht an die standardmäßig angeforderte Webcam-Auflösung gebunden.

Für FocusGuard ist es nicht notwendig, sämtliche vom Modell unterstützten Objektklassen zu verarbeiten. Im Unity-Inspector kann festgelegt werden, ob alle verfügbaren Klassen oder nur ausgewählte Objektklassen berücksichtigt werden sollen. Dadurch kann die Verarbeitung gezielt auf für FocusGuard relevante Critical Anchors beschränkt werden. Beispielsweise kann ausschließlich die Klasse *Person* berücksichtigt werden, wenn Personen als dynamische Critical Anchors untersucht werden sollen.

Für jeden Kandidaten wird innerhalb der aktivierten Klassen die höchste Konfidenz bestimmt. Kandidaten, deren Konfidenz unterhalb des konfigurierten **Confidence Threshold** liegt, werden verworfen. Dieser Grenzwert ist im Unity-Inspector einstellbar und beträgt standardmäßig 0,45.

YOLO kann für dasselbe reale Objekt mehrere ähnliche und stark überlappende Bounding Boxes erzeugen. Um solche Mehrfacherkennungen zu reduzieren, wird anschließend eine Non-Maximum Suppression (NMS) durchgeführt. Hierfür wird die Überschneidung zweier Bounding Boxes mithilfe der Intersection over Union (IoU) bewertet. Die IoU beschreibt das Verhältnis zwischen der Schnittfläche zweier Bounding Boxes und ihrer gemeinsamen Gesamtfläche. Ein Wert von 0 bedeutet, dass sich die Bounding Boxes nicht überschneiden, während ein Wert von 1 zwei vollständig übereinstimmende Bounding Boxes beschreibt.

Die Detektionen werden anhand ihrer Konfidenz bewertet. Überschreitet die Überlappung zweier Detektionen derselben Klasse den festgelegten IoU-Grenzwert, wird die Detektion mit der geringeren Konfidenz verworfen. Der IoU-Grenzwert ist ebenfalls im Unity-Inspector konfigurierbar und beträgt standardmäßig 0,45.

Eine mögliche Verbesserung der Bildaufbereitung besteht in einer seitenverhältniserhaltenden Vorverarbeitung mittels Letterboxing. Bei einem Ausgangsbild von 1280×720 Pixeln könnte das Bild beispielsweise proportional auf 640×360 Pixel verkleinert werden.

Um weiterhin die vom Modell erwarteten 640×640 Pixel zu erreichen, würden oberhalb und unterhalb des Bildes jeweils 140 Pixel Padding ergänzt. Die Proportionen der dargestellten Objekte blieben dadurch erhalten.

Eine solche Änderung würde gleichzeitig eine angepasste Rücktransformation der Bounding Boxes erfordern. Die zusätzlichen Randbereiche müssten von den Modellkoordinaten abgezogen und die verbleibenden Koordinaten anschließend mit dem verwendeten Skalierungsfaktor auf die ursprüngliche Framegröße übertragen werden. Die aktuelle Rückskalierung mit getrennten horizontalen und vertikalen Faktoren berücksichtigt zwar die tatsächliche Größe des Frames, kann die während der Inferenz durch die direkte Skalierung auf 640×640 entstehende geometrische Verzerrung jedoch nicht verhindern.

Abbildung 21 zeigt das Ergebnis der beschriebenen Verarbeitung anhand einer erkannten Person.

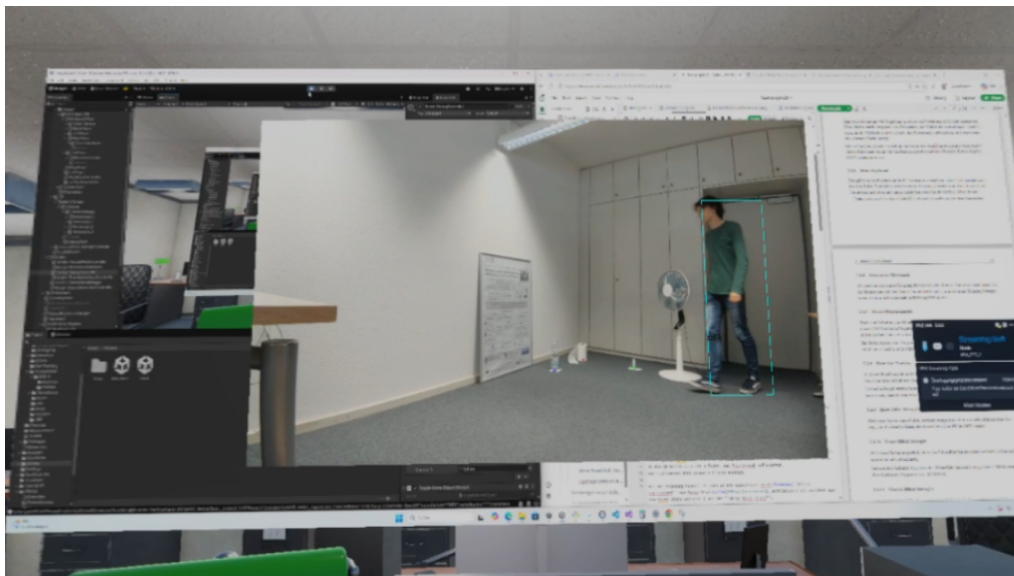


Abbildung 21: Erkennung einer Person im Bild der externen Webcam. Die Bounding Box beschreibt Position und Ausdehnung des erkannten Objekts im zweidimensionalen Kamerabild.

5.4.3 Detektionsergebnis und Konfiguration

Die nach der Filterung verbleibenden Objekte werden in einem `DetectionResult` zusammengefasst. Die Objekterkennung stellt dabei ausschließlich Informationen bereit, die unmittelbar aus dem Kamerabild bestimmt werden können. Für jedes erkannte Objekt werden die Objektklasse, die Konfidenz sowie Position und Größe der Bounding Box übergeben.

Information	Bedeutung
ClassId	Identifiziert die erkannte COCO-Objektklasse, beispielsweise eine Person.
Confidence	Gibt die Konfidenz des Modells für die erkannte Objektklasse an.
X, Y	Beschreiben die Position der Bounding Box innerhalb des Kamerabildes.
Width	Beschreibt die Breite der Bounding Box.
Height	Beschreibt die Höhe der Bounding Box.

Tabelle 1: Informationen eines erkannten Objekts innerhalb eines `DetectionResult`.

Ein `DetectionResult` enthält entsprechend eine Liste der in einem Frame erkannten Objekte. Die Objekterkennung beantwortet damit die Frage, welches Objekt mit welcher Sicherheit an welcher Position im zweidimensionalen Kamerabild erkannt wurde. Die Interpretation dieser Informationen im dreidimensionalen Raum erfolgt erst in der nachgelagerten räumlichen Integration.

Wesentliche Parameter der Detektionspipeline können direkt innerhalb des Unity-Inspectors verändert werden. Dadurch lässt sich die Objekterkennung an unterschiedliche Einsatzbedingungen anpassen, ohne Änderungen am Quellcode vorzunehmen.

Parameter	Funktion
Inference Backend	Auswahl des Backends für die Modellausführung; standardmäßig <code>GPUCompute</code> .
Detect All Classes / Enabled Classes	Verarbeitung aller unterstützten COCO-Klassen oder Beschränkung auf ausgewählte Objektklassen.
Confidence Threshold	Minimale Konfidenz einer weiterverarbeiteten Detektion; standardmäßig 0,45.
IoU Threshold	Grenzwert für die Non-Maximum Suppression überlappender Bounding Boxes; standardmäßig 0,45.
Inference Interval	Minimaler zeitlicher Abstand zwischen zwei Modelinferenzen; standardmäßig 0,5 Sekunden.
Webcam Device	Auswahl des verwendeten Kamerageräts.
Webcam Resolution / FPS	Angeforderte Auflösung und Bildrate; standardmäßig 1280×720 Pixel bei 30 FPS.

Tabelle 2: Zentrale konfigurierbare Parameter der Objekterkennung.

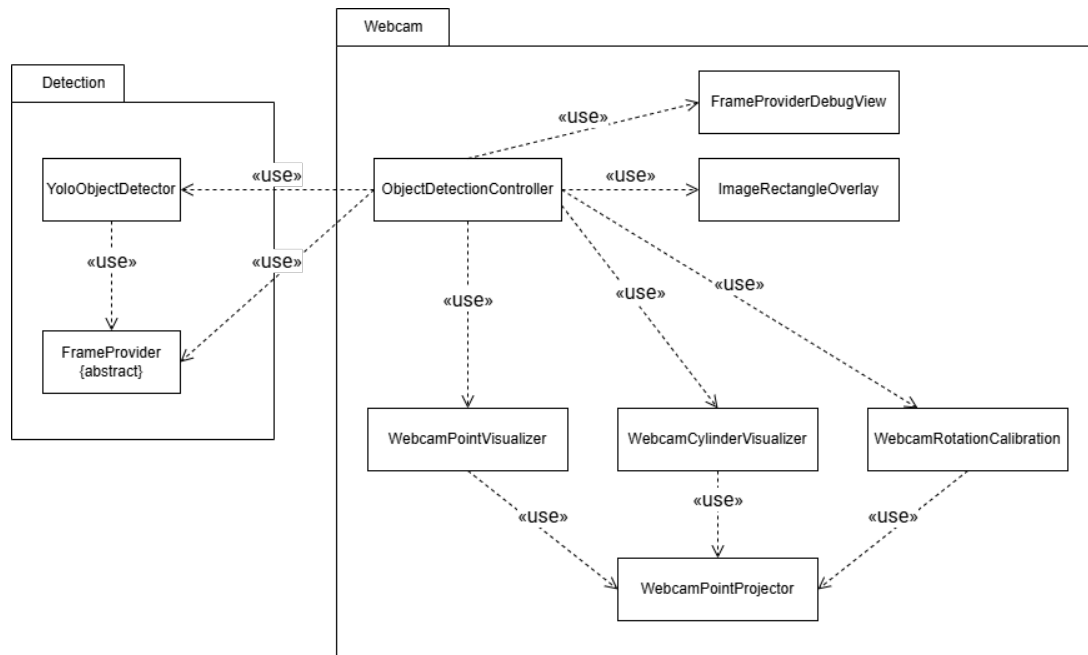


Abbildung 22: Paketdiagramm des Webcam-Moduls mit Schnittstellen zur Objekterkennung

5.5 Modul „Webcam“

Dieses Modul ist für die Verwendung der Ergebnisse der Objekterkennung zuständig. Zentrale Klasse ist der `ObjectDetectionController`, der zum einen die Liste der zu erkennenden Objektklassen und die zu verwendende Bildquelle enthält und diese an den `YoloObjectDetector` weitergibt, zum anderen die Ergebnisse der Objekterkennung abgreift und an andere Klassen weiterleitet, die diese darstellen (siehe Abb. 22).

Die Ergebnisse der Objekterkennung werden auf zwei Arten verwendet: erstens werden sie in das zweidimensionale Kamerabild eingezeichnet, sofern die Debugoption „Show Webcam“ aktiviert ist, zweitens werden die gefundenen Objekte in den dreidimensionalen Raum projiziert, um das Passthrough von Personen zu ermöglichen.

Für die erste Aufgabe ist die Klasse `FrameProviderDebugView` zuständig, die die Textur des Kameraausgabe auf ein Unity-Image-Objekt überträgt, sowie die separate Klasse `ImageRectangleOverlay`, die es ermöglicht, beliebige farbige Rechtecke über dem Bild zu zeichnen. Dabei werden sowohl die Outlines der gefundenen Objekte eingezeichnet, als auch Rechtecke, die die Kalibrierungspunkte repräsentieren.

Die wichtigere Aufgabe des Moduls ist die Projektion der gefundenen Objekte in den Raum. Die Klasse `WebcamCylinderVisualizer` erzeugt dafür an deren Position den Zylinder mit Passthrough. Dazu wird ein vorkonfiguriertes Zylinderobjekt kopiert, skaliert und an die richtige Stelle verschoben. Die Klasse `WebcamPointVisualizer` zeichnet analog zum `ImageRectangleOverlay` an der projizierten Position einen farbigen Rahmen,

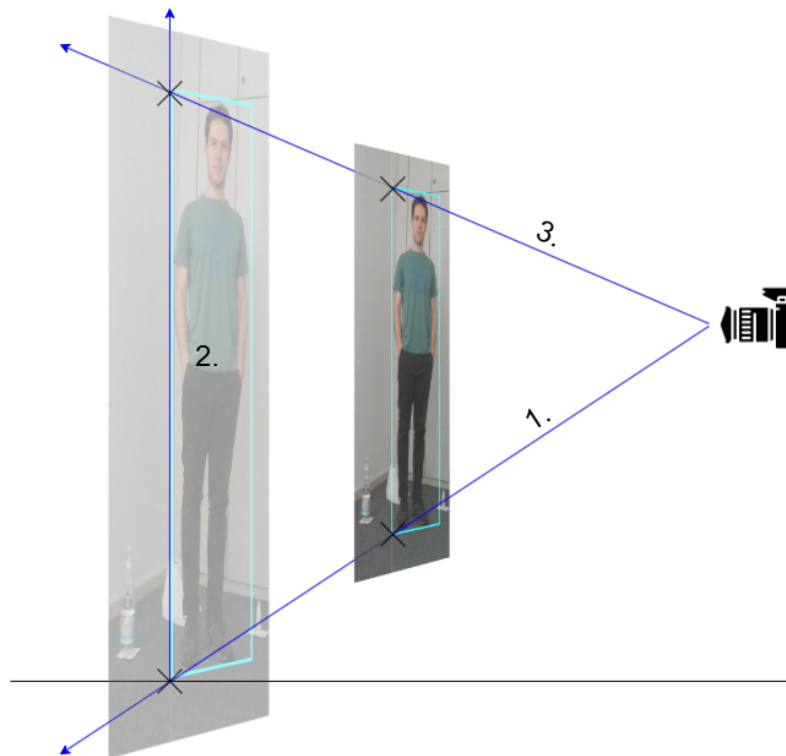


Abbildung 23: 2D-Darstellung der Projektion des oberen linken Punktes in den Raum

sofern wieder die Debug-Option „Show Camera“ aktiviert ist. Diese Klasse kann dabei auch einzelne Punkte projizieren, was wieder für die Kalibrierungspunkte verwendet wird. Zudem enthält die Klasse `WebcamRotationCalibration` den Code für die Kalibrierung der Webcam. Dabei werden - ausgehend von der aktuellen Rotation der Webcam - die Kalibrierungspunkte in den Raum projiziert und der Abstand der Punkte zu den bekannten Positionen der Kalibrierungsobjekte berechnet. Anschließend wird die Kamera iterativ entlang jeder Achse rotiert und dabei geschaut, in welcher Richtung der Abstand minimiert wird.

Die gemeinsame Logik zur Projektion von zweidimensionalen Kamerabildpunkten in den dreidimensionalen Raum ist in der Klasse `WebcamPointProjector` implementiert. Dieses Verfahren ist in Abb. 23 für den linken oberen Punkt eines erkannten Objektes schematisch dargestellt.

Ein zu projizierender Punkt wird - immer noch im zweidimensionalen Kamerabild - über drei Koordinaten beschrieben: Die x- und y-Koordinate eines Referenzpunktes im Kamerabild (in Pixeln) sowie dessen Höhe in Pixeln, d.h. wie viele Pixel über dem Referenzpunkt sich der eigentliche Punkt befindet. Vom Referenzpunkt nehmen wir dabei an, dass er sich auf dem Boden befindet. Handelt es sich um einen der oberen Punkte, wird der korrespondierende untere Punkt des Objektes als Referenzpunkt verwendet. Die unteren Punkte werden direkt als ihre eigenen Referenzpunkte mit der Höhe 0 verwendet, d.h. wir

gehen davon aus, dass die Punkte bereits auf dem Boden liegen.

Zur Projektion eines Punktes in den Raum wird zunächst der Schnittpunkt einer Linie, die durch den Referenzpunkt geht, mit der in der Kalibration bestimmten Bodenebene bestimmt (1. in Abb. 23). Durch diesen Schnittpunkt wird eine Ebene gelegt, die parallel zur Kamera liegt (2.). Nun muss nur noch der Schnittpunkt der Linie, die durch den ursprünglichen Punkt geht, mit dieser neuen Ebene bestimmt werden (3.), um den Punkt erfolgreich in die 3D-Welt zu projizieren.

Voraussetzung für die Genauigkeit dieser Projektion ist zum einen die korrekte Modellierung der Kamera hinsichtlich Position, Rotation und Field of View, aber auch die Annahme, dass sich der Referenzpunkt und damit auch das erkannte Objekt auf dem Boden befinden. Diese Annahme ist für Personen gut genug, selbst wenn sie auf einem Stuhl sitzen. Sollte sich ein Objekt nicht auf dem Boden befinden, würde die Projektion sich tendenziell zu weit hinter dem Objekt befinden. Befindet sich die Unterkante der Objektes sogar überhalb der Kamera, führt dies dazu, dass die Linie zum Referenzpunkt den Boden nicht mehr schneidet, weshalb das Objekt in diesem Fall gar nicht mehr projiziert werden kann.

6 Ausblick

In unserem Projekt konnten wir einen Prototypen entwickeln, der bereits viele Funktionen des FocusGuard-Konzeptes umsetzt. Dabei sind jedoch einige Probleme aufgetreten, die dafür sorgen, dass unser Prototyp noch nicht sinnvoll eingesetzt werden kann.

Insbesondere haben sich aus der Einschränkung auf die „VIVE Focus Vision“ als VR-Brille einige Probleme ergeben: Die Auflösung des Passthroughs und des virtuellen Bildschirms ist zu gering, um sinnvoll in der MR-Umgebung arbeiten zu können, und der fehlende Zugriff auf das Passthrough-Bild hat sowohl die Auswahl der Fokuseffekte eingeschränkt als auch zum Umweg bei der Objekterkennung über die Webcam geführt.

Ursprünglich wurde diese VR-Brille gewählt, da sie über Eye-Tracking verfügt. Die Qualität der Daten hat sich jedoch als ungenügend herausgestellt, um Mikrosakkaden zu erkennen, wie es im ursprünglichen Konzept vorgesehen war. Außerdem haben wir keinen geeigneten Weg gefunden, um aus der Messung der Pupillengröße auf einen Flow-Zustand des Benutzers zu schließen.

Es könnte sich daher anbieten, auf eine andere VR-Brille zu wechseln, die zwar über kein Eye-Tracking verfügt, dafür aber eine Objekterkennung über das Passthrough unterstützt. Dies ist beispielsweise bei der Meta Quest 3 der Fall [7]. Anstatt die Blickrichtung zur Ablenkungserkennung zu verwenden, müsste in diesem Fall die physische Orientierung der Kamera der VR-Brille genutzt werden. Dies wird aktuell bereits gemacht, wenn das Projekt ohne angeschlossene VR-Brille getestet wird.

Der genaue Ablauf des Fokuseffektes sollte noch einmal überarbeitet werden, insbesondere könnten auch Zwischenzustände auf dem Weg zu einer vollständigen virtuellen Umgebung entlang des „reality-virtuality continuum“ ermöglicht werden. Dazu sollten weitere Benutzerstudien durchgeführt werden, um die Auswirkungen des Systems auf die Benutzer besser zu untersuchen. Die konsequente Trennung zwischen der Datenmessung, Ablenkungserkennung und dem Anwenden des Fokuseffektes dürfte es ermöglichen, dass daraufhin Anpassungen einfach vorgenommen werden können.

Hinsichtlich des Problems, dass Bereiche am Rand des Bildschirms schwieriger zu erkennen sind, wurde die alternative Steuerung mit einer freien Bewegbarkeit des Monitors von den Benutzern als weniger intuitiv empfunden. Eine womöglich bessere Lösung, die z.B. von SteamVR verwendet wird, besteht darin, den Monitor nicht wie aktuell als flache Ebene in den Raum zu projizieren, sondern die Fläche zu krümmen, sodass der Bildschirm überall gleich weit vom Benutzer entfernt wäre. Diese Möglichkeit müsste implementiert und ausgetestet werden.

Erst kurz vor Ende des Projektes haben wir die Einstellung entdeckt, mit der Screenshots und Videos mit Passthrough aufgenommen werden könnten. Daher könnte womöglich

doch eine Objekterkennung mit der Focus Vision möglich sein. Allerdings wird eine API zum Bereitstellen des Kameravideos weiterhin nicht vom modernen VIVE OpenXR SDK ermöglicht, dafür ist eine solche Funktion anscheinend in der älteren VIVE Wave VR SDK vorhanden [2]. Diese scheint allerdings nur für einen Standalone-Betrieb auf der Brille über Android ausgelegt zu sein, daher würde der virtuelle Monitor nicht funktionieren. Mangels verfügbarer Zeit konnte diese Option nicht mehr getestet werden.

Literatur

- [1] HTC Corporation. Taking screenshots and screen recordings. Abgerufen am 17.09.2026. URL: https://www.vive.com/us/support/focusvision/category_howto/taking-screenshots-and-videos.html.
- [2] HTC Corporation. Wave VR Camera Texture, 2024. Abgerufen am 17.09.2026. URL: https://hub.vive.com/storage/docs/en-us/WaveVR_CameraTexture.html.
- [3] HTC Corporation. VIVE Focus Vision Specs, 2026. Abgerufen am 30.09.2026. URL: <https://www.vive.com/us/product/vive-focus-vision/specs/>.
- [4] Ralf Engbert and Reinhold Kliegl. Microsaccades uncover the orientation of covert attention. *Vision Research*, 43(9):1035–1045, 2003. URL: <https://www.sciencedirect.com/science/article/pii/S0042698903000841>, doi:10.1016/S0042-6989(03)00084-1.
- [5] International Organization for Standardization. Ergonomie in der Mensch-System-Interaktion – Teil 220: Prozesse zur Ermöglichung, Durchführung und Bewertung menschenzentrierter Gestaltung für interaktive Systeme in Hersteller- und Betreiberorganisationen. Standard, July 2020.
- [6] Chiara Krisam, Luca L. Bennardo, and Martin Feick. FocusGuard: Adaptive Transitions for Distraction-Aware Knowledge Work Using MR. In *International XR-Metaverse Conference*, 2026.
- [7] Meta. Multi Object Detection-Sample – Übersicht, 2026. Abgerufen am 17.09.2026. URL: <https://developers.meta.com/horizon/documentation/unity/unity-sample-camera-object-detection/>.
- [8] Paul Milgram, Haruo Takemura, Akira Utsumi, and Fumio Kishino. Augmented reality: a class of displays on the reality-virtuality continuum. In *Telemanipulator and Telepresence Technologies*, volume 2351, pages 282–292, December 1995. doi:10.1117/12.197321.
- [9] Marpa Studio. Office environment, 2025. Abgerufen am 17.09.2026. URL: <https://web.archive.org/web/20260306051312/https://assetstore.unity.com/packages/3d/environments/office-environment-329079>.